

SQLP 과목 III 튜닝 스타일 모의문제 · 7회차

SQL 자격검정 실전문제 3장 스타일 · 4지선다 · 총 20문항 · 원본 창작

1

데이터베이스의 저장 구조(세그먼트·블록)와 버퍼 캐시가 블록을 주고받는 상호작용에 대한 설명으로 가장 적절하지 않은 것은?

- ① 서버 프로세스가 필요한 데이터를 찾을 때 버퍼 캐시에 원하는 블록이 없으면, 디스크에서 그 블록이 아니라 조건을 만족하는 행(row)만 골라 캐시에 적재하므로, 한 블록에 불필요한 행이 많이 섞여 있어도 캐시에는 요청한 행만 올라온다.
- ② 세그먼트는 테이블·인덱스처럼 저장 공간을 차지하는 오브젝트가 데이터를 담는 단위이고, 세그먼트를 스캔한다는 것은 그 세그먼트에 속한 블록들을 읽어 버퍼 캐시로 올리는 과정이다.
- ③ I/O와 버퍼 캐시 관리는 블록 단위로 이루어지므로, 특정 행 하나만 필요하더라도 그 행이 속한 블록 전체가 캐시에 적재되고, 같은 블록 안의 다른 행은 이후 물리적 I/O 없이 캐시에서 참조된다.
- ④ Full Table Scan으로 대량의 블록을 읽어 들일 때, 오라클은 그 블록들이 자주 쓰이는 다른 블록을 캐시에서 밀어내지 않도록 LRU 리스트의 재사용되기 쉬운 쪽에 있어 관리하는 경향이 있다.

2

버퍼 캐시(SGA)를 우회해 디스크와 PGA 사이에서 곧바로 오가는 Direct Path I/O(direct path read·direct path write)와, 그때 관측되는 대기 이벤트에 대한 설명으로 가장 적절한 것은?

- ① Direct-Path Insert(/*+ append */)처럼 데이터를 고수위선(HWM) 위 새 블록에 곧바로 써 넣는 작업도, 기록에 앞서 그 블록을 버퍼 캐시에 올렸다가 DBWR가 내려쓰는 경로를 거치므로 일반 INSERT와 다를 바 없는 버퍼 캐시 경유 쓰기다.
- ② 여러 블록을 한 번의 I/O Call로 묶어 읽는 Multiblock I/O는 버퍼 캐시를 경유하는 경우에만 성립하는 개념이라, 버퍼 캐시를 우회하는 Direct Path Read는 블록을 하나씩만 집어 오는 Single Block I/O로 처리된다.
- ③ 정렬이나 해시 조인의 작업 영역이 PGA에 다 담기지 못해 임시 테이블스페이스로 내려 썼다가 되읽을 때는 버퍼 캐시를 우회하는 Direct Path I/O가 쓰이며, 내려쓰는 단계에서는 direct path write temp 대기 이벤트가 관측된다.
- ④ Full Table Scan을 직렬로 수행할 때는 버퍼 캐시를 경유하는 Multiblock I/O만 쓰이고, 버퍼 캐시를 우회하는 Direct Path Read는 병렬 쿼리에서만 나타나므로 직렬 Full Scan에서는 발동하지 않는다.

주차요금 정산 시스템의 야간 정산 배치가 주차권 40,000건을 순회하며 주차권마다 조건값을 문자열로 이어 붙인 리터럴 SQL 1건씩(총 40,000건, 서로 다른 SQL 문장)을 주차정산내역에 던진다. 8개의 배치 세션이 동시에 돌면서 응답이 72초로 늘고, CPU 사용률이 급등하며 대기 이벤트 상위에 library cache: mutex(라이브러리 캐시) 경합이 잡혔다. 아래는 이 배치 세션의 [비재귀 statements 합계]와 [재귀 statements 합계]를 나눠 집계한 TKPROF이다. 하드파싱과 재귀 파싱을 구분할 때, 다음 중 옳은 진단은? (단, 옵티마이저 통계는 최신이고 바인드 변수는 전혀 쓰지 않았으며, Shared Pool 크기는 여유가 충분해 에이징으로 인한 강제 재파싱은 없다.)

아래

OVERALL TOTALS FOR ALL NON-RECURSIVE STATEMENTS

Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	40000	24.000	30.000	0	0	0	0
Execute	40000	4.000	5.000	0	2000	0	0
Fetch	40000	2.000	3.000	200	118000	0	40000
Total	120000	30.000	38.000	200	120000	0	40000

Misses in library cache during parse: 37000

OVERALL TOTALS FOR ALL RECURSIVE STATEMENTS

Call	Count	CPU Time	Elapsed	Disk	Query	Current	Rows
Parse	50000	2.000	2.500	0	0	0	0
Execute	190000	16.000	18.000	0	700000	0	0
Fetch	240000	12.000	13.500	300	900000	0	190000
Total	480000	30.000	34.000	300	1600000	0	190000

Misses in library cache during parse: 800

- ① 비재귀 Parse 40,000회 중 37,000회가 라이브러리 캐시 미스(하드파싱율 92.5%)이고, 하드파싱이 부른 재귀 데이터 덱서너리 SQL까지 합치면 파싱에 든 CPU가 $24.0 + 30.0 = 54.0$ 초로 전체 CPU 60.0초의 90%다. 병목은 파싱 속에 있고 실행·계획·I/O 측과 독립이므로, 리터럴을 바인드 변수로 바꿔 하드파싱을 소프트파싱으로 돌리면 라이브러리 캐시 경합과 72초가 함께 줄어든다.
- ② 비재귀 Parse가 40,000회이니 애플리케이션이 서버를 40,000번 오간 것이므로, 파싱 왕복이 병목이다. 커넥션 풀링과 배열 인출 크기 상향으로 왕복 수를 줄이면 파싱 부하가 해소된다.
- ③ 재귀 Fetch의 Query 900,000 블록이 논리 읽기의 대부분이므로 병목은 재귀 SQL의 블록 소비다. DB 버퍼 캐시를 키워 이 900,000 블록의 논리 읽기를 줄이면 파싱 시간이 사라진다.
- ④ 하드파싱은 매번 최신 통계로 계획을 새로 만들어 실행 품질을 높이므로 성능에 이롭다. 바인드 변수로 바꾸면 bind peeking으로 계획이 나빠질 위험이 크니, 지금의 하드파싱을 유지하는 편이 안전하다.

4

EXPLAIN PLAN으로 적재해 DBMS_XPLAN.DISPLAY로 보는 예상(추정) 실행계획과, 실제 수행된 커서를 DBMS_XPLAN.DISPLAY_CURSOR로 보는 실제 실행계획이 행 수뿐 아니라 계획의 골격(노드 구성·순서)에서도 같릴 수 있는 이유에 대한 설명으로 가장 적절한 것은?

- ① 예상 계획과 실제 계획이 같리는 원인은 오직 각 단계의 카디널리티(행 수) 추정 오차뿐이며, 조인 순서나 액세스 경로 같은 계획의 골격은 두 계획에서 동일하게 유지된다.
- ② 적응적 실행계획(adaptive plan)이 걸리면 옵티마이저가 처음 세운 계획을 실행 도중 관측된 실제 행 수에 따라 다른 조인 방법으로 바꿀 수 있어, EXPLAIN PLAN이 보여 주는 예상 계획과 DISPLAY_CURSOR가 보여 주는 실제 수행 계획이 노드 구성 자체에서 달라질 수 있다.
- ③ DISPLAY_CURSOR는 방금 수행한 문장의 커서를 라이브러리 캐시에서 찾아 계획을 보여 주는 함수가 아니라, EXPLAIN PLAN이 PLAN_TABLE에 남긴 예상 계획을 대신 조회하는 함수이므로, 문장을 수행하지 않아도 실제 계획이 그대로 나온다.
- ④ 바인드 변수를 쓴 문장은 EXPLAIN PLAN이 그 바인드 값을 반영해 계획을 세우므로, DISPLAY로 본 예상 계획은 실제 그 값으로 수행된 커서의 계획과 노드 순서까지 정확히 일치한다.

5

음원 스트리밍 서비스에서 재생이력(약 1억 2,000만 건)을 청취자(약 800만 건), 곡마스터(약 5만 건)와 조인해 아이템별 재생 집계 3,000행을 뽑는 정산 SQL의 TKPROF이다. 재생이력 Full Scan은 몇 초 만에 끝날 것으로 봤는데 응답이 50초를 넘겨 느리다는 신고가 들어왔다. 아래 트레이스에 대한 해석으로 가장 적절한 것은? (단, 옵티마이저 통계는 최신이고 CPU·디스크 자원의 외부 경합은 없으며, 세 테이블 모두 인덱스가 없어 Full Scan + Hash Join으로만 수행된다.)

아 래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.008	0.008	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	4	9.000	50.000	260000	2000000	0	3000
Total	6	9.008	50.008	260000	2000000	0	3000
Rows	Row Source Operation						
3000	HASH GROUP BY (cr=2000000 pr=260000 pw=176000 time=48000000 us)						
120000000	HASH JOIN (cr=2000000 pr=260000 pw=176000 time=47000000 us)						
50000	TABLE ACCESS FULL 곡마스터 (cr=2000 pr=0 pw=0 time=60000 us)						
120000000	HASH JOIN (cr=1998000 pr=260000 pw=176000 time=44000000 us)						
8000000	TABLE ACCESS FULL 청취자 (cr=98000 pr=8000 pw=0 time=1500000 us)						
120000000	TABLE ACCESS FULL 재생이력 (cr=1900000 pr=76000 pw=0 time=9000000 us)						

- ① CPU Time 9초가 Elapsed 50초의 골격이라 CPU 바운드이므로, 병렬도(DOP)를 높여 CPU를 나눠 주면 50초가 개선된다.
- ② 논리 I/O 200만 중 26만(13%)이 디스크라 BCHR은 87.0%인데, 물리 읽기 260,000의 대부분(176,000, 약 68%)은 안쪽 HASH JOIN이 청취자 800만 행을 build하다 워크에어리어를 넘겨 temp로 흘린 spill이다(pw=176,000). 병목은 재생이력 스캔이 아니라 해시 조인의 디스크 spill이므로, PGA(해시 영역)를 키워 spill을 없애야 한다.
- ③ cr 200만 중 재생이력 Full Scan이 1,900,000(95%)으로 논리 읽기를 지배하므로 병목은 1억 2,000만 건 재생이력 스캔이다. 재생일시에 인덱스를 만들어 이 Full Scan을 줄이면 50초가 해소된다.
- ④ BCHR이 87%로 논리 대비 물리 읽기 비중이 높으니 디스크가 병목이다. DB 버퍼 캐시를 키워 물리 읽기 260,000을 캐시 적중으로 돌리면 약 41초의 대기가 사라진다.

피트니스 체인의 출입 정산 시스템에서 출입기록 테이블(약 4,800만 건)에는 결합 인덱스 출입_IX = (클럽코드, 출입일시)가 있다. 특정 클럽의 6월 입장 기록 중, 이용권 마스터(이용권_PK = 이용권번호)와 클럽 마스터(클럽_PK = 클럽코드)에 모두 존재하는 정합 건만 추리는 다음 SQL의 실행계획이다. 이 조회는 출입기록의 칼럼만 SELECT 하고, 두 마스터와의 조인은 존재 확인 용도이다.

```
SELECT e.출입번호, e.이용권번호, e.출입일시
FROM 출입기록 e, 이용권 p, 클럽 s
WHERE e.클럽코드 = 'C017'
AND e.출입일시 >= DATE '2026-06-01'
AND e.출입일시 < DATE '2026-07-01'
AND e.출입구분 = '입장'
AND p.이용권번호 = e.이용권번호
AND s.클럽코드 = e.클럽코드;
```

이 실행계획에 대한 설명으로 가장 적절한 것은?

아 래					
Id	Pid	Operation	(Cost	Card	Bytes)
0	-	SELECT STATEMENT	(742	790	33970)
1	0	NESTED LOOPS	(742	790	33970)
2	1	NESTED LOOPS	(740	810	30780)
3	2	TABLE ACCESS BY INDEX ROWID 출입기록	(737	810	26730)
4	3	INDEX RANGE SCAN 출입_IX	(8	3100	)
5	2	INDEX UNIQUE SCAN 이용권_PK	(0	1	9)
6	1	INDEX UNIQUE SCAN 클럽_PK	(0	1	5)

- ① 선두 칼럼 클럽코드의 등치 조건과 두 번째 칼럼 출입일시의 범위 조건이 출입_IX의 인덱스 액세스 조건이 되어 INDEX RANGE SCAN(Id 4)으로 처리되고, 인덱스에 없는 출입구분='입장'은 Id 3 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 걸러진다. 이용권_PK·클럽_PK는 존재만 확인하는 인덱스 유니크 스캔이라 테이블 액세스가 뒤따르지 않는다.
- ② 클럽코드·출입일시·출입구분 세 조건이 함께 출입_IX의 인덱스 액세스 조건으로 처리되어 INDEX RANGE SCAN(Id 4)의 스캔 시작·종료 지점을 좁힌다.
- ③ 선두 칼럼 클럽코드 조건 없이 출입일시 범위 조건만 주어져도 출입_IX는 같은 비용의 INDEX RANGE SCAN으로 동일하게 처리된다.
- ④ Id 5·Id 6의 인덱스 유니크 스캔 뒤에는 반드시 TABLE ACCESS BY INDEX ROWID가 따라와 이용권·클럽 테이블 블록을 읽어야 조인이 완성된다.

공항 수하물 추적 시스템에서 수하물스캔이력(약 6,000만 건, 테이블 총 블록 약 60,000)을 도착공항 코드 인덱스로 검색해 특정 공항의 스캔 300,000행을 인덱스 순서로 읽어 내려받는 SQL의 TKPROF이다. 인덱스로 잘 타는데도 응답이 55초를 넘긴다. 개발자는 "Fetch가 여러 번이라 인출 왕복이 병목"이라 보고 배열 크기를 더 키우려 한다. 아래 트레이스로 볼 때, 배열 크기(Array Processing)와 클러스터링 팩터 두 지표로 진단할 때 실제 병목은 무엇인가? (단, 옵티마이저 통계는 최신이고, 애플리케이션은 이미 배열 인출을 사용하며 인덱스는 도착공항 코드 단일 컬럼으로 구성돼 있다.)

아래							
Call	Count	CPU Time	Elapsed Time	Disk	Query	Current	Rows
Parse	1	0.002	0.002	0	0	0	0
Execute	1	0.000	0.000	0	0	0	0
Fetch	301	6.000	55.000	40000	293100	0	300000
Total	303	6.002	55.002	40000	293100	0	300000
Rows	Row Source Operation						
300000	TABLE ACCESS BY INDEX ROWID 수하물스캔이력 (cr=293100 pr=40000 pw=0 time=52000000 us)						
300000	INDEX RANGE SCAN 수하물스캔이력_도착공항_IDX (cr=2100 pr=300 pw=0 time=900000 us)						

- ① Fetch가 301회나 발생한 것이 병목이다. 배열 크기가 작아 300,000행을 나누어 나르는 인출 왕복이 55초를 만들었으므로, 배열 크기를 더 키우면 왕복이 줄어 해소된다.
- ② 배열 크기는 $300,000 \div (301 - 1) = 1,000$ 으로 이미 커서 Fetch가 301회뿐이라 왕복은 병목이 아니다. 반면 테이블 액세스 $cr\ 291,000 \div ROWID\ 300,000 = 0.97$ 로 클러스터링 팩터 밀도가 최댓값 1에 근접(완전 정렬 시 0.001의 970배)이라, 인덱스로 뽑은 행마다 흩어진 블록을 랜덤 액세스한 것이 병목이다. 인덱스 키 순서로 테이블을 재정렬하거나 커버링 인덱스로 테이블 액세스를 없애야 한다.
- ③ 논리 읽기 $cr\ 293,100$ 이 과다한 것이 병목이므로, 인덱스를 재생성하거나 컬럼을 보강해 인덱스 스캔의 논리 읽기를 줄이면 55초가 개선된다.
- ④ 물리 읽기 $pr\ 40,000$ 이 55초 대기를 만든 근본 원인이므로, DB 버퍼 캐시를 키워 이 40,000 블록을 캐시 적중으로 돌리면 랜덤 액세스가 사라진다.

인덱스만 읽고 끝내는 인덱스 전용 스캔(커버링)과, 인덱스 전체를 읽는 두 방식 — Index Full Scan과 Index Fast Full Scan — 에 대한 설명으로 가장 적절하지 않은 것은?

- ① 인덱스 전용 스캔(커버링)은 쿼리가 참조하는 컬럼이 인덱스 안에 모두 들어 있을 때 테이블(TABLE ACCESS BY INDEX ROWID)을 되짚지 않고 인덱스만 읽어 결과를 내는 것으로, ROWID로 테이블 블록을 임의 접근하는 Random 액세스를 없애 조회 부담을 덜는다.
- ② 선두 컬럼에 걸 조건이 없어 Range Scan을 쓰지 못하더라도, 조회에 필요한 컬럼이 인덱스에 다 있으면 옵티마이저는 인덱스 전체를 리프 연결 순서로 훑는 Index Full Scan으로 테이블을 건드리지 않고 처리할 수 있다.
- ③ Index Fast Full Scan은 인덱스 세그먼트를 리프의 논리적 연결이 아니라 물리적 저장 순서대로 Multiblock I/O로 읽어 대량 처리에 유리하고 병렬로도 수행되지만, 결과가 인덱스 키 순서로 정렬되어 나오지는 않는다.
- ④ Index Fast Full Scan은 리프 블록을 논리적 연결 순서를 따라 한 블록씩 Single Block I/O로 읽어 결과가 인덱스 키 순서로 정렬되고, Index Full Scan은 세그먼트를 물리적 저장 순서로 Multiblock I/O로 읽어 정렬을 보장하지 않는다.

9

자식 테이블의 외래키(FK) 컬럼에 인덱스를 둘지 판단하는 설계와, 그에 따른 동시성·부하의 맞교환에 대한 설명으로 가장 적절하지 않은 것은?

- ① 자식 테이블의 외래키 컬럼에 인덱스가 없으면, 부모 테이블의 키 값을 삭제하거나 변경할 때 오라클은 참조 무결성을 확인하려 자식 테이블에 넓은 범위의 Lock을 걸고 자식을 전체 스캔하는 경향이 있어, 자식 테이블의 동시 DML이 막히는 경향이 생길 수 있다.
- ② 외래키 인덱스는 이런 부모-자식 참조 무결성 검사 시의 Lock·스캔 부담을 줄여 줄 뿐 아니라, 부모와 자식을 잇는 조인에서 자식 쪽으로 들어가는 진입 경로로도 쓰여 조회에도 도움이 된다.
- ③ 그럼에도 외래키 인덱스 역시 하나의 인덱스이므로 자식 테이블에 DML이 일어날 때마다 갱신되는 유지 부하를 더하며, 그 인덱스로 얻는 동시성·조회 이득과 이 유지 비용을 견주어 생성 여부를 판단해야 한다.
- ④ 외래키 인덱스가 부모 키 변경 시의 자식 테이블 Lock과 전체 스캔을 완화해 주므로, 외래키 인덱스는 조회 성능뿐 아니라 자식 테이블의 INSERT·UPDATE·DELETE 부하까지 낮추어, 인덱스가 늘수록 DML이 가벼워지는 대표적 사례가 된다.

10

채용 포털의 통계 배치가 채용공고 테이블(진행 중 공고 약 3만 건)과 지원내역 테이블(약 9,700만 건)을 공고번호로 조인해 채용직군별 접수 건수를 집계한다. 네 후보 SQL은 SELECT 목록·WHERE 조건·GROUP BY가 모두 같고, 조인 힌트만 서로 다르다. 아래 실행계획을 낸 SQL로 가장 적절한 것은?

아래

Id	Operation	Name
0	SELECT STATEMENT	
1	SORT GROUP BY	
* 2	HASH JOIN	
* 3	TABLE ACCESS FULL	채용공고
* 4	TABLE ACCESS FULL	지원내역

① 아래 SQL

```
SELECT /*+ LEADING(a c) USE_HASH(c) */
      c.채용직군, COUNT(*)
FROM   채용공고 c, 지원내역 a
WHERE  a.공고번호 = c.공고번호
      AND c.채용상태 = '진행중'
      AND a.지원상태 = '접수'
GROUP BY c.채용직군;
```

② 아래 SQL

```
SELECT /*+ LEADING(c a) USE_NL(a) */
      c.채용직군, COUNT(*)
FROM   채용공고 c, 지원내역 a
WHERE  a.공고번호 = c.공고번호
      AND c.채용상태 = '진행중'
      AND a.지원상태 = '접수'
GROUP BY c.채용직군;
```

③ 아래 SQL

```
SELECT /*+ LEADING(c a) USE_HASH(a) */
      c.채용직군, COUNT(*)
FROM   채용공고 c, 지원내역 a
WHERE  a.공고번호 = c.공고번호
      AND c.채용상태 = '진행중'
      AND a.지원상태 = '접수'
GROUP BY c.채용직군;
```

④ 아래 SQL

```
SELECT /*+ LEADING(c a) USE_MERGE(a) */
      c.채용직군, COUNT(*)
FROM   채용공고 c, 지원내역 a
WHERE  a.광고번호 = c.광고번호
      AND c.채용상태 = '진행중'
      AND a.지원상태 = '접수'
GROUP BY c.채용직군;
```

11

'서브쿼리에 해당하지 않는 행'을 골라내는 NOT IN 서브쿼리와 NOT EXISTS 서브쿼리, 그리고 이들이 풀리는 안티(anti) 조인에 대한 설명으로 가장 적절한 것은?

- ① NOT IN 서브쿼리와 NOT EXISTS 서브쿼리는 '해당하지 않는 행을 고른다'는 같은 의미이므로, 서브쿼리 결과에 NULL이 섞여 있어도 두 쿼리는 같은 결과 집합을 돌려주며 서로 바꿔 써도 된다.
- ② 서브쿼리가 돌려주는 값 중에 NULL이 하나라도 있으면 NOT EXISTS는 바깥 행을 하나도 통과시키지 못해 결과가 비지만, NOT IN은 NULL을 건너뛰고 매칭 없는 행을 정상적으로 돌려준다.
- ③ 서브쿼리 컬럼이 NOT NULL로 선언돼 있어도 NOT IN은 안티 조인으로 변환될 수 없어, 옵티마이저는 이를 정렬·해시가 없는 단순 필터로만 처리한다.
- ④ NOT IN 서브쿼리 컬럼에 NULL이 존재할 수 있으면 $x \neq \text{NULL}$ 이 참도 거짓도 아닌 UNKNOWN이 되어 어떤 바깥 행도 조건을 만족하지 못하므로, NOT IN은 NOT EXISTS와 결과가 달라질 수 있고, 옵티마이저도 이 NULL 의미 차이 때문에 두 형태를 같은 안티 조인으로 취급하지 않는다.

12

옵티마이저의 쿼리 변환 중 조건절 이행, 서브쿼리 Unnesting, OR-Expansion이 만들어 내는 액세스 경로에 대한 설명으로 가장 적절하지 않은 것은?

- ① 서브쿼리를 늘 Unnesting할 수 있는 것은 아니어서, 변환이 불가능하거나 이득이 없다고 판단되면 옵티마이저는 서브쿼리를 조인으로 풀지 않고 바깥 행마다 평가하는 필터(FILTER) 오퍼레이션으로 남겨 둔다.
- ② OR-Expansion은 OR로 묶인 조건을 여러 개의 단일 조건 브랜치로 분리해 각각을 UNION ALL로 잇는 변환으로, 브랜치마다 서로 다른 인덱스를 액세스 조건으로 쓸 수 있게 해 준다.
- ③ 조건절 이행으로 유도된 새 조건은 원래는 없던 인덱스 액세스 경로를 열어 줄 수 있어, 옵티마이저는 그 경로를 후보에 넣고 비용으로 원래 경로와 비교한다.
- ④ 서브쿼리 Unnesting과 OR-Expansion은 둘 다 쿼리를 여러 조각으로 나눠 각각 조인·인덱스를 고르게 하는 변환이므로, 서브쿼리에 OR 조건이 들어 있으면 Unnesting은 반드시 OR-Expansion을 거친 뒤에야 적용된다.

13

기술문서 번역 시스템의 번역요청(약 1,000만 건)은 처리상태 칼럼이 극도로 편중돼 있다 — '번역완료' 약 970만(97%), '번역중' 약 29만(2.9%), '연체' 약 1만(0.1%). 이 칼럼에는 도수분포 히스토그램이 수집돼 있고, 인덱스 ix_처리상태가 있다. 애플리케이션은 아래 SQL을 바인드 변수 :b1 로 여러 상태값에 대해 반복 실행한다. 상태값별 조치가 섞여 실행된 뒤 라이브러리 캐시(v\$sql)의 자식 커서를 요약한 결과가 함께 주어졌다. 바인드 변수·히스토그램·커서 공유에 대한 설명으로 가장 적절하지 않은 것은?

아래

```
SELECT 번역번호, 회원번호, 번역일자
FROM   번역요청
WHERE  처리상태 = :b1;
```

[v\$sql - 번역요청 조회 커서(처리상태 = :b1), 편중 칼럼 + 히스토그램]

SQL_ID	CHILD	IS_BIND_SENSITIVE	IS_BIND_AWARE	PLAN_HASH_VALUE	EXECUTIONS	
9hq3ub71ctxm4	0	Y	N	2716355091	40	← 첫 peek 계획 재사용
9hq3ub71ctxm4	1	Y	Y	3182904677	7	← 희소값('연체'류)
9hq3ub71ctxm4	2	Y	Y	2716355091	18	← 다수값('번역완료'류)

* PLAN_HASH_VALUE 2716355091 = TABLE ACCESS FULL, 3182904677 = INDEX RANGE SCAN + TABLE ACCESS BY ROWID

- ① 처리상태에 히스토그램이 있어 옵티마이저가 값의 편중을 인지하므로, 바인드 변수로 실행하면 파싱 시점에 첫 실행의 바인드 값을 엿보아(Bind Peeking) 그 값 기준으로 실행계획을 세운다. 그래서 자식 0의 IS_BIND_SENSITIVE가 Y로 표시됐다.
- ② 만약 첫 실행이 희소값 '연체'로 peek돼 인덱스 레인지 스캔 계획이 커서에 굳었다면, 이어 '번역완료'(약 970만)로 실행돼도 같은 커서를 재사용해 대량 건에 비효율적인 인덱스 액세스가 유지될 수 있다 — 첫 peek 계획이 굳는 문제다.
- ③ 바인드 변수는 SQL 텍스트를 하나로 고정해 커서를 공유하므로, 처리상태 값의 분포와 무관하게 옵티마이저가 항상 단일한 최적 실행계획을 산출하며 어떤 값으로 실행해도 똑같이 효율적이다.
- ④ 오라클의 Adaptive Cursor Sharing은 편중 칼럼에 바인드가 쓰이면 커서를 bind-sensitive로 표시하고 실행 통계가 쌓이면 bind-aware로 전환해, 바인드 값 구간별로 별도 자식 커서를 만든다. 요약표의 자식 1·2가 서로 다른 PLAN_HASH_VALUE로 갈린 것이 그 증거다.

14

옵티마이저가 카디널리티를 추정할 때 쓰는 NDV와 히스토그램(도수분포·높이균형), 그리고 히스토그램이 없을 때의 균등 분포 가정에 대한 설명으로 가장 적절한 것은?

- ① NDV(서로 다른 값의 개수)가 큰 컬럼일수록 값이 고르게 퍼져 있다는 뜻이므로, 그런 컬럼에는 히스토그램을 수집할 실익이 크고, NDV가 작은 컬럼은 이미 분포가 평탄해 히스토그램이 필요 없다.
- ② 히스토그램으로 특정 값의 선택도가 정확해지면 그 값으로 조건을 건 쿼리는 히스토그램이 없을 때보다 더 낮은 비용으로 산정되어 실행이 빨라지므로, 히스토그램 수집은 곧 해당 조건의 실행 속도를 높이는 수단이다.
- ③ 값의 종류가 버킷 수보다 적어 값마다 버킷을 하나씩 줄 수 있으면 각 값의 빈도를 그대로 담은 도수분포(frequency) 히스토그램이, 값의 종류가 버킷 수보다 많으면 여러 값을 한 버킷에 묶어 분포를 근사하는 높이균형(height-balanced) 히스토그램이 만들어지며, 어느 쪽이든 히스토그램이 없을 때의 균등 분포 가정보다 편중된 값의 선택도를 정밀하게 추정하게 해 준다.
- ④ 히스토그램의 버킷 수를 늘릴수록 옵티마이저가 인식하는 그 컬럼의 NDV도 함께 커지므로, 버킷을 촘촘히 잡으면 서로 다른 값의 개수가 늘어 카디널리티 추정이 정밀해진다.

관세청 통관시스템의 수입신고(약 8억 건)는 신고일자 기준 일별 RANGE 파티션 테이블이다. 야간 배치는 당일 신고분(약 400만 건)을 비파티션 임시 테이블 수입신고_당일에 대량 적재한 뒤, 이를 대상 파티션과 교환(EXCHANGE PARTITION) 하는 방식으로 반영한다(아래 DDL·DML). 두 테이블 모두 NOLOGGING 속성이며, 데이터베이스는 FORCE LOGGING이 아니다. 이 대량 적재·교환 방식에 대한 설명으로 가장 적절한 것은?

아 래

— 임시 테이블(비파티션) : 대상 파티션과 구조·칼럼 동일, NOLOGGING
CREATE TABLE 수입신고_당일 (...) NOLOGGING;

— ① 당일분 대량 적재 (Direct Path + 병렬)
ALTER SESSION ENABLE PARALLEL DML;
INSERT /*+ APPEND PARALLEL(t, 4) */ INTO 수입신고_당일 t
SELECT * FROM 외부신고_수신 s;
COMMIT;

— ② 로컬 인덱스 생성 후 대상 파티션과 교환
ALTER TABLE 수입신고
EXCHANGE PARTITION p_20260718 WITH TABLE 수입신고_당일
INCLUDING INDEXES WITHOUT VALIDATION;

- ① INSERT /*+ APPEND */는 세그먼트 HWM 위에 새 블록을 직접 기록하는 Direct Path 적재이고, 대상 테이블이 NOLOGGING이며 FORCE LOGGING도 아니므로 이 적재의 데이터 리두가 최소화(minimal logging)된다. 이후 EXCHANGE PARTITION은 데이터를 물리 이동하지 않고 디렉터리에서 세그먼트를 맞바꾸는 메타데이터 연산이라 대용량이라도 순식간에 끝난다.
- ② 병렬 DML은 PARALLEL 힌트만 주면 세션 설정과 무관하게 INSERT 부분이 병렬로 수행되므로, ALTER SESSION ENABLE PARALLEL DML 문장은 실제 병렬 적재에 영향을 주지 않는 관례적 선언에 불과하다.
- ③ Direct Path 적재가 진행되는 동안에도 다른 세션은 수입신고_당일에 자유롭게 일반 INSERT/UPDATE를 수행할 수 있으며, 적재 세션은 커밋 전까지 그 테이블에 배타적 잠금을 걸지 않는다.
- ④ EXCHANGE PARTITION에 WITHOUT VALIDATION을 주면 검증 비용을 없애는 동시에, 임시 테이블에 대상 파티션 범위를 벗어난 행이 섞여 있어도 오라클이 교환 시점에 이를 안전하게 걸러 올바른 파티션으로 자동 재배치한다.

폐기물 수거 관리의 수거접수는 접수일자(DATE)로 반기 RANGE 파티셔닝하고, 그 아래를 배출자번호로 HASH 서브파티셔닝(SUBPARTITIONS 4) 한 복합 파티션 테이블이다(아래 DDL). 각 선택지는 SELECT SUM(수거금액) FROM 수거접수 WHERE ... 형태의 조회가 어떤 파티션·서브파티션을 접근하는지를 설명한다. Partition Pruning이 설명대로 작동한다고 보기 가장 적절하지 않은 것은?

아래

```
CREATE TABLE 수거접수 (
  수거번호    NUMBER,
  접수일자    DATE,
  배출자번호  NUMBER,
  수거등급    VARCHAR2(10),
  수거금액    NUMBER
)
PARTITION BY RANGE (접수일자)
SUBPARTITION BY HASH (배출자번호) SUBPARTITIONS 4
(
  PARTITION p_2026h1 VALUES LESS THAN (DATE '2026-07-01'),
  PARTITION p_2026h2 VALUES LESS THAN (DATE '2027-01-01'),
  PARTITION p_max    VALUES LESS THAN (MAXVALUE)
);
```

- ① WHERE 접수일자 >= DATE '2026-07-01' 은 RANGE 파티션 프루닝으로 p_2026h2·p_max만 접근하되, 배출자번호 조건이 없으므로 그 두 파티션 각각의 해시 서브파티션 4개는 전부 스캔한다.
- ② WHERE 배출자번호 = 8801234 만 주면 RANGE로는 좁힐 수 없어 세 파티션(p_2026h1·h2·max)이 대상이지만, 각 파티션 안에서는 배출자번호를 해시 매핑한 서브파티션 1개씩만 접근하는 서브파티션 프루닝이 걸린다.
- ③ WHERE 접수일자 = DATE '2026-08-10' AND 배출자번호 = 8801234 는 RANGE로 p_2026h2 한 파티션, HASH로 그 안의 서브파티션 1개까지 좁혀 최종 단일 서브파티션만 읽는다.
- ④ WHERE SUBSTR(TO_CHAR(배출자번호),1,3) = '880' 은 배출자번호 앞 3자리로 조회하므로, 해시 서브파티션 프루닝이 작동해 880으로 시작하는 회원이 매핑된 서브파티션만 골라 읽는다.

17

가스 사용량 정산 야간 배치가 사용량집계(약 3억 6천만 건), 공급계약(약 4,300만 건), 단가적용(약 4,300만 건) 세 테이블을 계약번호로 조인해 계약번호별 정산액을 집계한다. 세 테이블은 모두 계약번호 기준 해시 64 파티션으로 나뉘어 있다. 아래 병렬 실행계획에서 세 테이블이 병렬 서버(Slave) 사이에 어떻게 분배되어 조인되는지에 대한 설명으로 가장 적절한 것은?

아래

Id	Operation	Name	Pstart	Pstop	TQ	IN-OUT
0	SELECT STATEMENT					
1	PX COORDINATOR					
2	PX SEND QC (RANDOM)	:TQ10000			Q1,00	P->S
3	HASH GROUP BY				Q1,00	PCWP
4	PX PARTITION HASH ALL		1	64	Q1,00	PCWC
* 5	HASH JOIN				Q1,00	PCWP
* 6	HASH JOIN				Q1,00	PCWP
7	TABLE ACCESS FULL	사용량집계	1	64	Q1,00	PCWP
8	TABLE ACCESS FULL	공급계약	1	64	Q1,00	PCWP
9	TABLE ACCESS FULL	단가적용	1	64	Q1,00	PCWP

- ① 대형 사용량집계만 파티션 단위로 각 서버가 자기 조각을 읽고, 공급계약·단가적용은 사용량집계의 파티션 경계에 맞춰 PX SEND PARTITION (KEY)로 재분배하는 부분(partial) 파티션 와이즈 조인이다.
- ② 세 테이블이 계약번호로 동일하게 64개 해시 파티셔닝돼 있어, 각 병렬 서버가 대응하는 파티션 집합만 로컬로 조인한다. 조인 입력 사용량집계·공급계약·단가적용(Id 7·8·9)이 같은 Q1,00·같은 PX PARTITION HASH ALL(Id 4) 아래에서 PX SEND/PX RECEIVE 없이 읽혀 재분배가 일어나지 않는 (full) 파티션 와이즈 조인이며, Id 3 HASH GROUP BY도 파티션 키(계약번호) 기준이라 서버 로컬로 끝난다.
- ③ 세 입력을 조인 키 계약번호의 해시로 각각 재분배(hash-hash)한 뒤 각 서버가 자기 버킷 범위의 행만 조인하며, 그 재분배가 Id 3에서 일어난다.
- ④ 소형 공급계약·단가적용을 병렬 서버 전체에 복제(broadcast)하고 대형 사용량집계만 재분배 없이 파티션 단위로 읽는 broadcast 분배다.

18

GROUP BY를 처리하는 두 방식(SORT GROUP BY·HASH GROUP BY)과 인덱스를 이용한 소트 대체에 대한 설명으로 가장 적절하지 않은 것은?

- ① SORT GROUP BY는 그룹 키로 정렬하며 집계해 결과가 그룹 키 순서를 띠지만, HASH GROUP BY는 해시 테이블에 그룹별로 누적해 집계하므로 결과가 그룹 키 순서로 정렬되어 나오지 않는다.
- ② 정렬 그룹핑과 해시 그룹핑은 모두 PGA work area에서 수행되며, 집계 대상이 work area에 다 담기지 못하면 Temp 테이블스페이스를 쓰는 디스크 처리로 넘어가 I/O 부하가 커진다.
- ③ GROUP BY 대상 컬럼에 인덱스가 있거나 하면 옵티마이저는 그 인덱스로 그룹핑을 처리해, SORT GROUP BY든 HASH GROUP BY든 집계 오퍼레이션이 실행계획에서 사라진다.
- ④ 인덱스의 정렬 순서로 소트가 생략되는 것은 정렬로 그룹을 모으는 SORT GROUP BY에 해당하는 이야기이며, 해시 매칭으로 그룹을 모으는 HASH GROUP BY는 애초에 정렬을 쓰지 않아 인덱스 정렬 순서로 대체되는 대상이 아니다.

19

탄소배출권 거래대장 배출권에서 참여기업 'M-77'의 잔여 배출권(초기 30,000)를 두 트랜잭션이 아래 순서로 접근한다. TX2는 같은 SELECT를 t1과 t3에서 두 번 수행하는데, 그 사이 t2에서 TX1이 5,000을 적립·커밋한다. TX2의 격리수준이 READ COMMITTED일 때와 SERIALIZABLE일 때, t3의 두 번째 SELECT가 반환하는 잔여 배출권을 순서대로 나열한 것으로 적절한 것은? (단, 오라클이며 두 격리수준 외의 설정은 조정하지 않았다.)

아래

시각	TX1	TX2 (READ COMMITTED / SERIALIZABLE)
t1		SELECT 잔여 FROM 배출권 WHERE 참여기업='M-77'; (첫 번째 읽기)
t2	UPDATE 배출권 SET 잔여 = 잔여 + 5000 WHERE 참여기업='M-77'; (30000 → 35000) COMMIT;	
t3		SELECT 잔여 FROM 배출권 WHERE 참여기업='M-77'; (두 번째 읽기)
t4		COMMIT;
초기 잔여 = 30000		

- ① READ COMMITTED 35,000 / SERIALIZABLE 30,000
- ② READ COMMITTED 30,000 / SERIALIZABLE 35,000
- ③ READ COMMITTED 35,000 / SERIALIZABLE 35,000
- ④ READ COMMITTED 30,000 / SERIALIZABLE 30,000

20

물류센터 피킹시스템(SQL Server)의 피킹작업 테이블을 두 세션이 엇갈려 갱신 중이다. 세션 61은 주문 5001행을 먼저 잠근 뒤 주문 6002행을 갱신하려 하고, 세션 74는 반대로 6002를 먼저 잠근 뒤 5001을 갱신하려 한다. 이때 sys.dm_tran_locks를 조회한 결과가 아래와 같다(RCSI 미사용, 기본 READ COMMITTED). 이 잠금 상태와 SQL Server의 처리에 대한 해석으로 가장 적절하지 않은 것은?

아래

[sys.dm_tran_locks - 피킹작업 두 행에 엇갈린 잠금 (교착 직전)]

request_session_id	resource_type	resource_description	request_mode	request_status
61	KEY	(주문 5001 행)	X	GRANT
61	KEY	(주문 6002 행)	X	WAIT
74	KEY	(주문 6002 행)	X	GRANT
74	KEY	(주문 5001 행)	X	WAIT

* KEY = 클러스터형 인덱스 키(행) 잠금, X = 배타 잠금
* GRANT = 잠금 획득 보유, WAIT = 잠금 요청 대기

- ① 세션 61은 주문 5001 행에 X 잠금을 GRANT로 보유한 채 6002 행의 X 잠금을 WAIT로 요청하고, 세션 74는 반대로 6002를 보유하고 5001을 요청하므로 서로가 서로를 기다리는 순환 대기가 형성됐다.
- ② SQL Server의 잠금 에스컬레이션은 테이블 잠금 경합이 감지되면 이를 페이지→행 잠금으로 잘게 낮춰 동시성을 높이는 방향으로 작동하므로, 이 교착도 곧 행 단위로 분해되며 저절로 해소된다.
- ③ SQL Server의 락 모니터는 이런 순환 대기를 기본 주기로 자동 감지해, 롤백 비용이 적은 쪽을 교착 희생자(deadlock victim)로 선정하고 오류 1205와 함께 그 트랜잭션을 롤백해 상대 세션을 진행시킨다.
- ④ 만약 두 세션이 같은 순서(둘 다 5001을 먼저, 그다음 6002)로 접근했다면 순환이 생기지 않아, 한 세션이 앞 세션의 커밋까지 잠깐 대기하는 단순 블로킹에 그쳤을 것이다.

정답 및 해설

■ 정답 모아보기

1. ①	2. ③	3. ①	4. ②	5. ②
6. ①	7. ②	8. ④	9. ④	10. ③
11. ④	12. ④	13. ③	14. ③	15. ①
16. ④	17. ②	18. ③	19. ①	20. ②

문제 1. 정답 ①

데이터베이스가 디스크와 메모리 사이에서 데이터를 주고받는 최소 단위는 블록입니다. 행 하나를 원하든 여러 행을 원하든, 오가는 화폐 단위는 언제나 블록입니다.

저장 구조 : 세그먼트(테이블 · 인덱스) ⊃ 익스텐트 ⊃ 블록
I/O 단위 : 블록 (행 단위가 아님)
버퍼 캐시 : 디스크에서 읽은 '블록'을 통째로 올려 두는 공간

한 행만 필요할 때 → 그 행이 든 '블록 전체'가 캐시로 올라옴
→ 같은 블록의 다른 행은 이후 캐시에서 재사용(물리 I/O 0)

①은 이 단위를 정반대로 뒤집었습니다. "디스크에서 행만 골라 캐시에 적재한다"고 했지만, 캐시에 적재되는 것은 행이 아니라 블록입니다. 필요한 행이 한 건이어도 그 행이 속한 블록 전체가 올라오며, 그렇기 때문에 한 블록에 불필요한 행이 많이 섞여 있으면(넓은 로우, 낮은 밀도) 같은 데이터를 읽는 데 더 많은 블록 I/O가 드는 것입니다. "요청한 행만 올라온다"는 서술은 블록 단위 I/O라는 기본 전제를 거꾸로 세운 것입니다.

②·③·④는 옳습니다. 세그먼트 스캔이 곧 블록을 읽어 캐시로 올리는 과정이라는 점(②), 블록 단위 적재로 같은 블록의 다른 행이 물리 I/O 없이 재사용된다는 점(③), 대량 스캔 블록을 LRU의 밀려나기 쉬운 쪽에 얹어 캐시 오염을 줄인다는 점(④)이 모두 저장 구조와 버퍼 캐시 상호작용에 부합합니다.

선택지	판정	이유
①	×	디스크와 캐시가 주고받는 단위는 블록입니다. 한 행만 필요해도 블록 전체가 캐시에 올라오며, "행만 골라 적재한다"는 것은 블록 단위 I/O를 뒤집은 서술입니다
②	○	세그먼트는 오브젝트가 데이터를 담는 단위이고, 세그먼트 스캔은 그에 속한 블록들을 읽어 버퍼 캐시로 적재하는 과정입니다
③	○	I/O·캐시 관리가 블록 단위라, 한 행만 필요해도 블록 전체가 적재되고 같은 블록의 다른 행은 물리 I/O 없이 캐시에서 참조됩니다
④	○	Full Table Scan의 대량 블록은 유용한 블록을 밀어내지 않도록 LRU의 재사용되기 쉬운 쪽에 얹어 캐시 오염을 줄이는 방향으로 관리됩니다

■ 이 문제의 핵심

1. 디스크↔캐시의 거래 단위는 블록입니다. 행 하나를 원해도 블록 전체가 오갑니다.
2. 세그먼트 스캔 = 그 세그먼트의 블록을 읽어 캐시로 올리는 것입니다.
3. 블록 단위 적재 덕에 같은 블록의 다른 행은 이후 물리적 I/O 없이 캐시에서 재사용됩니다.
4. 대량 스캔 블록은 LRU의 밀려나기 쉬운 쪽에 얹어 캐시 오염을 줄입니다.

■ 한 줄 정리 데이터베이스가 디스크와 캐시 사이에서 주고받는 단위는 블록이므로 한 행만 필요해도 블록 전체가 캐시에 올라오는데, "행만 골라 캐시에 적재한다"는 서술은 이 블록 단위 I/O를 정반대로 뒤집은 것입니다.

문제 2. 정답 ③

Direct Path I/O는 버퍼 캐시(SGA)를 거치지 않고 디스크와 수행 프로세스의 PGA 사이에서 블록을 곧바로 주고받는 경로입니다. 읽기와 쓰기 양쪽에 다 있고, 각기 다른 대기 이벤트로 관측됩니다.

경로	방향	버퍼 캐시	대표 대기 이벤트
Direct Path Read	읽기	우회	direct path read (임시영역 되읽기: direct path read temp)
Direct Path Write	쓰기	우회	direct path write

(정렬/해시 스킵: direct path write temp)

정렬·해시 작업영역이 PGA를 넘침

- 임시 테이블스페이스로 '내려쓰기' = direct path write temp
- 뒤에 다시 '되읽기' = direct path read temp

③은 이 관계를 정확히 짚었습니다. 정렬·해시 조인의 워크에어리어가 PGA에 다 못 담기면 임시 테이블스페이스로 스킵하는데, 이 스킵 I/O는 캐시를 우회하는 Direct Path I/O이고 내려쓰는 단계에서 **direct path write temp** 대기가 잡힙니다.

나머지는 모두 Direct Path의 동작 경계를 뭉개 서술입니다. ①은 캐시를 우회하는 Direct-Path Insert를 캐시 경유 쓰기로, ②는 캐시 우회 읽기를 Single Block으로만, ④는 Direct Path Read를 병렬 전용으로 좁혔습니다.

선택지	판정	이유
①	×	Direct-Path Insert는 버퍼 캐시를 우회해 HWM 위 새 블록에 곧바로 기록합니다. "캐시에 올렸다가 DBWR가 내려쓴다"는 것은 일반(conventional) 경로와의 경계를 뭉개 서술입니다
②	×	Direct Path Read도 인접한 여러 블록을 한 Call에 묶어 읽는 Multiblock I/O입니다. 다만 그 블록을 캐시가 아닌 PGA로 적재할 뿐이며, Single Block으로 처리되는 것이 아닙니다
③	○	정렬·해시 작업영역이 PGA를 넘쳐 임시 테이블스페이스로 스킵할 때는 캐시를 우회하는 Direct Path I/O가 쓰이고, 내려쓰기 단계에서 direct path write temp 대기가 관측됩니다
④	×	Direct Path Read는 병렬 쿼리 전용이 아닙니다. 직렬 수행이라도 큰 세그먼트를 훑을 때는 직렬 Direct Path Read가 발동할 수 있어, "직렬 Full Scan에서는 안 나온다"는 경계는 틀립니다

■ 이 문제의 핵심

1. Direct Path I/O = 버퍼 캐시 우회, 디스크↔PGA 직행 경로입니다. 읽기·쓰기 모두 존재합니다.
2. 정렬·해시 스킵은 임시 테이블스페이스로의 Direct Path Write이며, 내려쓰기는 **direct path write temp**로 잡힙니다.
3. Direct Path Read도 Multiblock입니다. 여러 블록을 한 Call에 묶되 캐시 대신 PGA로 적재할 뿐입니다.
4. Direct-Path Insert는 HWM 위에 직접 기록하고, 직렬 Full Scan도 Direct Path Read가 발동할 수 있어 병렬 전용이 아닙니다.

■ 한 줄 정리 Direct Path I/O는 캐시를 우회하는 읽기·쓰기 경로이고 정렬·해시 스킵의 임시영역 쓰기는 **direct path write temp**로 관측되는데, 이를 캐시 경유·Single Block·병렬 전용으로 좁힌 서술들은 Direct Path의 동작 경계를 뭉개 것입니다.

문제 3. 정답 ①

리터럴 SQL은 문장마다 텍스트가 달라 라이브러리 캐시에서 공유되지 못하므로, Parse마다 미스가 나 하드파싱이 됩니다. 먼저 하드파싱율을 셉니다.

$$\begin{aligned}\text{하드파싱율} &= \text{라이브러리 캐시 미스(비재귀)} \div \text{비재귀 Parse 수} \\ &= 37,000 \div 40,000 = 92.5\%\end{aligned}$$

40,000번의 파싱 중 37,000번이 하드파싱입니다. 하드파싱은 계획을 새로 만들기 위해 데이터 딕셔너리를 뒤지는데, 이 딕셔너리 조회가 재귀 statement로 잡힙니다. 그래서 재귀 블록의 Execute·Fetch가 대량으로 발생합니다. 이제 파싱 비중을 봅니다.

$$\begin{aligned}\text{파싱에 든 CPU} &= \text{비재귀 Parse CPU} + \text{재귀 전체 CPU(하드파싱이 부른 딕셔너리 조회)} \\ &= 24.0 + 30.0 = 54.0\text{초} \\ \text{전체 CPU} &= \text{비재귀 30.0} + \text{재귀 30.0} = 60.0\text{초} \\ \text{파싱 비중} &= 54.0 \div 60.0 \times 100 = 90.0\%\end{aligned}$$

$$\text{실작업(비재귀 Execute+Fetch) CPU} = 4.0 + 2.0 = 6.0\text{초 (전체의 10\%)}$$

CPU 60초 중 54초(90%)가 파싱이고, 실제 주차자산내역을 읽고 나른 실작업은 6초(10%)뿐입니다. 응답 72초의 정체는 파싱 속에 있습니다.

축 A: 파싱(하드파싱 37,000회) — 라이브러리 캐시 경합·CPU 90%의 정체
축 B: 실행/계획/I/O — 실작업 CPU 6초로 이미 가벼움

동시 8세션이 저마다 하드파싱하면 라이브러리 캐시 mutex를 서로 다투므로, 파싱 속을 줄이지 않는 한 실행·I/O를 손대도 72초는 그대로.

처방은 리터럴을 바인드 변수로 바꿔 하드파싱을 소프트파싱으로 돌리는 것입니다. 같은 SQL을 공유하면 미스가 사라져 딕셔너리 재귀 조회도, 라이브러리 캐시 경합도 함께 빠집니다. ①이 파싱 속과 실행/계획/I/O 속을 옹게 분리했습니다.

선택지	판정	이유
①	○	하드파싱율 $37,000/40,000 = 92.5\%$, 파싱 비중 $54.0/60.0 = 90\%$ 로 병목이 파싱 축임을 짚고, 바인드 변수로 하드→소프트 전환해 라이브러리 캐시 경합을 줄이는 처방이 정확합니다
②	×	Parse 40,000회는 서버 내부 파싱 횟수이지 클라이언트 왕복 수가 아닙니다. 파싱 비용 축을 왕복 축으로 오인한 것으로, 커넥션 풀링·배열 인출은 90%가 파싱인 CPU를 겨냥하지 못합니다
③	×	재귀 Query 900,000은 하드파싱이 부른 디렉터리 조회의 논리 읽기라, 파싱을 없애면 함께 사라지는 종속 결과입니다. 버퍼 캐시(I/O 축)를 키워도 하드파싱 자체(파싱 축)는 그대로여서 원인을 다른 축에 오귀속한 것입니다
④	×	파싱 비용 축과 계획 품질 축은 별개입니다. 통계가 최신이면 소프트파싱도 같은 계획을 재사용하므로, 하드파싱을 유지해 얻는 계획 품질 이득은 없고 90%의 파싱 CPU만 낭비됩니다

■ 이 문제의 핵심

- 하드파싱율 = 라이브러리 캐시 미스 ÷ Parse 수. 여기서는 $37,000/40,000 = 92.5\%$ 로, 리터럴 SQL이라 거의 모든 파싱이 하드파싱입니다.
- 하드파싱은 데이터 디렉터리 재귀 조회를 유발합니다. 재귀 블록의 Execute·Fetch·Query는 하드파싱의 종속 부하이지 독립된 병목이 아닙니다.
- 파싱 비중 = (비재귀 Parse + 재귀 전체) CPU ÷ 전체 CPU = $54/60 = 90\%$. 실작업은 6초(10%)뿐이라, 병목은 파싱 축입니다.
- 파싱 축은 실행/계획/I/O·왕복 축과 독립입니다. 축을 뒤섞으면 버퍼 캐시·커넥션 풀링·계획 품질로 헛짚습니다.
- 처방은 바인드 변수입니다. 하드→소프트 전환이 라이브러리 캐시 경합과 디렉터리 재귀 조회를 동시에 없앱니다.

■ 한 줄 정리 파싱 CPU가 전체의 90%(54/60초)이고 하드파싱율이 92.5%(37,000/40,000)이므로, 72초 병목은 버퍼 캐시·왕복·계획 품질이 아니라 파싱 축이며 바인드 변수로 하드파싱을 소프트파싱으로 돌려야 풀린다.

문제 4. 정답 ②

두 도구는 서로 다른 곳에서, 서로 다른 시점의 계획을 읽어 옵니다. 그래서 행 수 추정치만이 아니라 계획의 골격까지 어긋날 수 있습니다.

DISPLAY ← EXPLAIN PLAN이 PLAN_TABLE에 적재한 '예상' 계획
(문장 미수행, 바인드 값은 엿보지 못함)

DISPLAY_CURSOR ← 라이브러리 캐시에 올라온 '실제 수행된 커서'의 계획
(문장을 실제 실행해야 커서가 생김)

두 계획이 '골격'까지 갈리는 계기

- 바인드 값 미반영(EXPLAIN PLAN) vs bind peeking(실제 파싱)
- 적응적 실행계획: 실행 도중 실측 행 수를 보고 조인 방법을 교체

②가 옳습니다. 적응적 실행계획은 옵티마이저가 처음 세운 계획을 실행 초기에 실제로 흘러온 행 수와 비교해, 예컨대 NL 조인을 해시 조인으로 갈아탈 수 있게 해 둔 장치입니다. 이 경우 **EXPLAIN PLAN**이 뽑은 예상 계획과 실제 커서의 최종 계획은 카디널리티 숫자만이 아니라 조인 방법이라는 노드 구성 자체에서 달라집니다.

①·③·④는 예상·실제 두 계획의 출처와 성격의 경계를 뭉갯습니다. 차이를 행 수로만 좁히려(①),

DISPLAY_CURSOR의 출처를 PLAN_TABLE로 바꾸거나(③), **EXPLAIN PLAN**이 바인드를 엿본다고(④) 본 것이 모두 사실과 다릅니다.

선택지	판정	이유
①	×	두 계획은 행 수 추정 오차뿐 아니라 조인 순서·엑세스 경로 같은 골격까지 갈릴 수 있습니다. 차이를 카디널리티로만 좁힌 것은 발문의 전제와 어긋납니다
②	○	적응적 실행계획은 실행 도중 실측 행 수에 따라 조인 방법을 교체하므로, 예상 계획과 실제 커서 계획이 노드 구성 자체에서 달라질 수 있습니다
③	×	DISPLAY_CURSOR 는 라이브러리 캐시의 '실제 수행된 커서'를 읽습니다. PLAN_TABLE의 예상 계획을 대신 읽는 것이 아니며, 커서가 있으려면 문장이 실제로 수행돼 있어야 합니다
④	×	EXPLAIN PLAN 은 바인드 값을 엿보지(bind peeking) 못하고 미지수로 취급합니다. 그래서 실제 값으로 수행된 커서의 계획과 오히려 갈릴 수 있어, "노드 순서까지 일치"는 틀립니다

■ 이 문제의 핵심

- DISPLAY**(예상)는 PLAN_TABLE, **DISPLAY_CURSOR**(실제)는 라이브러리 캐시에서 계획을 읽어 옵니다. 출처와 시점이 다릅니다.

2. 두 계획은 행 수뿐 아니라 골격까지 같릴 수 있습니다. 바인드 미반영과 적응적 실행계획이 대표적 계기입니다.
3. 적응적 실행계획은 실행 도중 실측 행 수를 보고 조인 방법을 바꾸므로, 예상↔실제가 노드 구성에서 달라집니다.
4. **EXPLAIN PLAN**은 bind peeking을 못 합니다. 바인드가 있으면 예상 계획이 실제와 어긋날 수 있습니다.

▪ 한 줄 정리 예상 계획(PLAN_TABLE)과 실제 커서 계획(라이브러리 캐시)은 바인드 미반영·적응적 실행계획 때문에 행 수를 넘어 노드 구성까지 같릴 수 있으며, 이를 "차이는 행 수뿐"·"출처가 같다"·"바인드를 엮는다"로 좁힌 서술들은 두 계획의 경계를 뚫은 것입니다.

문제 5. 정답 ②

먼저 BCHR로 논리 대비 물리 읽기를 봅니다.

BCHR = ((Query + Current) - Disk) / (Query + Current) × 100
 = (2,000,000 + 0 - 260,000) / 2,000,000 × 100
 = 1,740,000 / 2,000,000 × 100 = 87.0%

디스크 물리 읽기 비중 = 260,000 / 2,000,000 × 100 = 13.0%

다음은 시간의 정체입니다.

CPU 비중 = 9.000 / 50.000 × 100 = 18.0% → 나머지 82%가 대기

Elapsed - CPU = 50.000 - 9.000 = 41.0초 ← 기다린 시간

응답의 82%가 대기입니다. 대기가 난 곳을 Row Source로 찾되, cr(논리)과 pr·pw(물리·temp)를 분리해서 봐야 합니다.

테이블 스캔 물리 읽기 = 재생이력 76,000 + 청취자 8,000 = 84,000
 안쪽 HASH JOIN pr = 260,000, pw = 176,000
 → 스캔이 읽은 84,000을 빼면 260,000 - 84,000 = 176,000이 spill 재읽기
 물리 읽기 260,000 중 spill 비중 = 176,000 / 260,000 × 100 ≈ 67.7%

pw=176,000은 결정적입니다. 안쪽 HASH JOIN이 **청취자 800만 행**을 build하다 해시 영역을 넘겨 temp로 spill했고, 그 176,000 블록을 다시 읽느라 물리 읽기의 약 68%가 발생했습니다. 시간도 이 노드에 몰립니다.

재생이력 Full Scan time = 9.0초 ← 큰 테이블이지만 멀티블록이라 빠름
 안쪽 HASH JOIN time = 44.0초 ← spill I/O로 정체된 지점

cr은 재생이력 Full Scan(1,900,000, 95%)이 지배하지만, 그 스캔은 9초 만에 끝났습니다. 50초를 만든 것은 논리 읽기가 아니라 해시 조인의 디스크 spill(pw=176,000)입니다. 따라서 처방은 PGA(해시 영역) 확대이며, ②가 BCHR과 병목 지목을 모두 옳게 했습니다.

선택지	판정	이유
①	×	CPU 9초는 Elapsed 50초의 18%뿐이라 CPU 바운드가 아닙니다. 82%(41초)가 spill I/O 대기이므로 DOP 상향은 원인을 잘못 짚은 것입니다
②	○	BCHR (2,000,000-260,000)/2,000,000 = 87.0%가 정확하고, pw=176,000과 물리 읽기의 68%가 안쪽 HASH JOIN의 spill임을 짚어 PGA를 처방한 것이 맞습니다
③	×	cr 95%가 재생이력 Full Scan인 것은 사실이나 그 스캔은 time 9초로 빠릅니다. 50초 정체는 논리 읽기가 아니라 pw=176,000의 spill이므로, 인덱스로 cr을 줄이자라는 수치는 맞되 병목을 오귀속한 처방입니다
④	×	물리 읽기 260,000의 68%(176,000)는 temp spill 재읽기라 DB 버퍼 캐시로는 캐시되지 않습니다. BCHR 87%를 근거로 버퍼 캐시를 키워도 spill 대기 41초는 남아, 대기의 원인을 캐시로 오귀속한 것입니다

▪ 이 문제의 핵심

1. BCHR = ((Query+Current)-Disk)/(Query+Current)×100. 여기서는 (2,000,000-260,000)/2,000,000 = 87.0%입니다.
2. cr(논리)과 pr·pw(물리·temp)를 분리해야 합니다. cr은 재생이력 Full Scan(95%)이 지배해도, 시간을 만든 건 pw=176,000의 해시 조인 spill입니다.
3. **pw>0**은 해시 영역 spill의 신호입니다. 물리 읽기 260,000 중 176,000(68%)이 spill 재읽기이고, 테이블 스캔 물리는 84,000뿐입니다.
4. build input이 큰 노드(청취자 800만 행)가 워크에어리어를 넘기면 spill합니다 — 병목은 build 쪽 조인이지만 probe 쪽 큰 테이블 스캔이 아닙니다.
5. 처방은 PGA(해시 영역) 확대입니다. DOP·인덱스·버퍼 캐시는 41초의 spill 대기를 겨냥하지 못합니다.

▪ 한 줄 정리 BCHR 87%까지 옳게 읽어도, 물리 읽기 260,000의 68%가 pw=176,000의 해시 조인 spill이라는 사실을 놓치면 병목을 CPU·재생이력 스캔·버퍼 캐시로 오귀속하게 된다.

문제 6. 정답 ①

플랜의 세 노드에서 나온 Card 값을 위에서 아래로 훑으면 인덱스가 어디까지 걸러 주는지가 보입니다.

INDEX RANGE SCAN 출입_IX Card 3,100 ← 인덱스로 훑은 건수
TABLE ACCESS BY INDEX ROWID 출입기록 Card 810 ← 테이블 필터를 거친 뒤 남은 건수
NESTED LOOPS (최종) Card 790 ← 두 마스터 존재 확인까지 마친 건수

출입_IX는 (클럽코드, 출입일시) 순서입니다. WHERE에는 클럽코드 = 'C017'(등치)와 출입일시 범위가 있으므로, 선두 칼럼 등치 + 두 번째 칼럼 범위까지가 인덱스로 연속해 훑을 수 있는 구간입니다. 그 결과가 INDEX RANGE SCAN의 3,100건입니다.

출입구분은 인덱스 칼럼이 아닙니다. 인덱스만으로는 걸러낼 수 없어, ROWID로 테이블 블록을 읽어 온 Id 3 TABLE ACCESS BY INDEX ROWID 단계에서 필터로 처리됩니다. 그래서 3,100건이 이 단계를 지나며 810건으로 줄어듭니다.

3,100 (인덱스 액세스: 클럽코드=, 출입일시 범위)
└─ 테이블 필터(출입구분='입장') 적용 → 810
└─ 이용권_PK · 클럽_PK 존재 확인(유니크 스캔) → 790 (최종)

Id 5·Id 6은 이용권번호·클럽코드가 마스터에 있지만 확인하는 조인입니다. SELECT는 **출입기록** 칼럼만 뽑으므로 이용권·클럽 테이블에서 읽어 올 칼럼이 없어, INDEX UNIQUE SCAN에서 끝나고 테이블 액세스로 내려가지 않습니다(플랜에 두 테이블의 TABLE ACCESS 노드가 없다는 것이 그 표식입니다). ①이 이 구조를 정확히 서술합니다.

선택지	판정	이유
①	○	클럽코드(등치)·출입일시(범위)는 인덱스 액세스 조건, 출입구분은 인덱스에 없어 Id 3 테이블 필터 — Card가 3,100→810으로 줄고, 두 유니크 스캔은 테이블 액세스 없이 존재만 확인하는 흐름과 일치합니다
②	×	출입구분은 출입_IX 구성 칼럼이 아니므로 인덱스 액세스 조건이 될 수 없습니다. 세 조건 전부가 스캔 범위를 좁혔다면 INDEX RANGE SCAN Card가 이미 810이었을 텐데 실제로는 3,100입니다 — 테이블 필터 뒤편 인덱스 액세스로 끌어올린 서술입니다
③	×	선두 칼럼 클럽코드가 등치로 고정돼야 출입일시 범위가 인덱스의 연속 구간이 됩니다. 선두 조건이 빠지면 스캔 시작점을 잡지 못해 같은 비용의 INDEX RANGE SCAN이 성립하지 않습니다
④	×	이용권·클럽 칼럼을 SELECT 하지 않아 읽어 올 칼럼이 없으므로, Id 5·Id 6은 INDEX UNIQUE SCAN에서 끝납니다. 플랜에 두 테이블의 TABLE ACCESS 노드가 아예 없어 테이블 액세스가 뒤따르지 않습니다

■ 이 문제의 핵심

1. 결합 인덱스는 선두 칼럼 등치 + 다음 칼럼 범위까지가 액세스 조건. 여기서는 클럽코드(=)·출입일시(범위)가 INDEX RANGE SCAN 구간을 만들어 3,100건을 냅니다.
2. 인덱스에 없는 칼럼은 테이블 필터. 출입구분은 ROWID로 테이블을 읽은 뒤 Id 3 TABLE ACCESS BY INDEX ROWID 단계에서 걸러져 810건으로 줄어듭니다.
3. 선두 칼럼이 빠지면 RANGE SCAN 효율이 무너집니다. 선두 등치가 스캔 시작점을 고정해 줘야 두 번째 칼럼 범위가 연속 구간이 됩니다.
4. 필요한 칼럼이 없으면 인덱스 유니크 스캔에서 끝납니다. Id 5·Id 6은 존재 확인만 하므로 테이블 액세스로 내려가지 않습니다 — 플랜에 해당 TABLE ACCESS 노드가 없다는 것이 근거입니다.

■ 한 줄 정리 클럽코드 등치와 출입일시 범위는 **출입_IX** 인덱스 액세스 조건이 되어 3,100건을 내고, 인덱스에 없는 출입구분은 Id 3 테이블 필터로 810건까지 줄이며, 칼럼을 뽑지 않는 두 마스터 조인은 INDEX UNIQUE SCAN에서 끝나 테이블 액세스가 없다.

문제 7. 정답 ②

개발자의 가설("왕복이 병목")부터 배열 크기로 검증합니다. 마지막 한 번은 "더 없음(no-more-data)" 인출이라 -1 합니다.

ArraySize = 총 Rows ÷ (Fetch 횟수 - 1)
= 300,000 ÷ (301 - 1)
= 300,000 ÷ 300 = 1,000 (행/Fetch)

한 번에 1,000행씩 담아 오니 30만 행을 나르는 데 Fetch는 301회뿐입니다. 배열 크기는 이미 충분히 커서 왕복은 병목이 아닙니다 — 개발자의 가설은 여기서 기각됩니다. 그러면 55초는 어디서 났을까요. 클러스터링 팩터를 봅니다. 인덱스 cr은 테이블 노드에 누적되므로 테이블 랜덤 블록만 떼어 냅니다.

테이블 랜덤 블록(cr) = TABLE 노드 cr - INDEX 노드 cr
= 293,100 - 2,100 = 291,000
CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수
= 291,000 ÷ 300,000 = 0.97 ← 최댓값 1에 근접(최악)
완전 정렬 시 밀도 = 테이블 총 블록 ÷ 테이블 총 행

$= 60,000 \div 60,000,000 = 0.001$ (블록당 1,000행)
 약화 배수 = $0.97 \div 0.001 \approx 970$ 배

CF 밀도 0.97은 거의 1 — 인덱스로 뽑은 행 하나하나가 저마다 다른 테이블 블록에 흩어져 있어, 300,000행을 읽으며 291,000번의 테이블 랜덤 블록 액세스가 일어났다는 뜻입니다. 완전히 정렬됐다면 블록당 1,000행씩 담겨 약 300블록이면 될 것을, 291,000블록을 방문한 셈입니다. 시간·논리 읽기가 모두 이 랜덤 액세스에 몰립니다.

INDEX RANGE SCAN cr = 2,100 (전체 cr 293,100의 0.7%) ← 인덱스는 가벼움
 TABLE 랜덤 액세스 cr = 291,000 (99.3%) ← 논리 읽기의 대부분
 TABLE 랜덤 액세스 time = 52초 (전체 55초의 약 95%) ← 시간의 대부분
 CPU 비중 = $6.0 / 55.0 \times 100 \approx 10.9\%$ → 대기 바운드

두 지표가 함께 말해 줍니다: 배열 크기(1,000)는 정상이라 왕복이 아니고, CF 밀도(0.97)가 최악이라 테이블 랜덤 액세스가 병목입니다. 처방은 배열 크기 상향이 아니라, 인덱스 키 순서로 테이블을 재정렬(CF 개선)하거나 필요한 컬럼을 인덱스에 담아 커버링으로 테이블 액세스를 없애는 것입니다. ②가 두 지표를 옳게 결합했습니다.

선택지	판정	이유
①	×	ArraySize = $300,000 / (301 - 1) = 1,000$ 으로 배열 크기가 이미 커 Fetch는 301회뿐입니다. 왕복은 병목이 아니므로 배열 크기를 더 키워도 291,000번의 테이블 랜덤 액세스는 그대로 남아, 개발자 가설을 그대로 따른 오귀속입니다
②	○	ArraySize 1,000으로 왕복 가설을 기각하고, CF 밀도 $291,000 / 300,000 = 0.97$ (최댓값 1에 근접)로 테이블 랜덤 액세스를 병목으로 지목한 뒤 CF 개선·커버링을 처방한 것이 정확합니다
③	×	cr 293,100이 큰 것은 맞지만 그 99.3%(291,000)는 인덱스가 아니라 테이블 랜덤 액세스입니다. INDEX 스캔 cr은 2,100뿐이라 인덱스를 재생성·보강해도 291,000번의 테이블 블록 방문은 줄지 않아, 원인을 인덱스로 오귀속한 것입니다
④	×	pr 40,000을 캐시로 돌리면 물리 읽기는 줄어도, CF 밀도 0.97이 만든 291,000번의 논리적 테이블 랜덤 블록 방문 자체는 남습니다. 병목의 원인(나쁜 CF)을 물리 읽기로 오귀속한 처방입니다

■ 이 문제의 핵심

1. ArraySize = 총 Rows ÷ (Fetch 횟수 - 1). 여기서는 $300,000 / (301 - 1) = 1,000$ 으로, 배열 크기가 이미 커 왕복(301회)은 병목이 아닙니다 — "Fetch가 많다"는 인상만으로 왕복을 탓하면 안 됩니다.
2. CF 밀도 = 테이블 랜덤 블록(cr) ÷ ROWID 수. 테이블 cr은 인덱스 cr을 뺀 291,000이고, $291,000 / 300,000 = 0.97$ 로 최댓값 1에 근접합니다(완전 정렬 시 $60,000 / 60,000,000 = 0.001$ 의 970배).
3. CF 밀도가 1에 가까우면 인덱스로 뽑은 행마다 서로 다른 블록에 흩어져 있어 테이블 랜덤 액세스가 폭증합니다 — 시간(52초)·논리 읽기(99.3%)가 모두 이 노드에 몰립니다.
4. 두 지표를 함께 봐야 진단이 유일해집니다: 배열 크기가 정상임을 확인해 왕복 가설을 기각하고, CF로 테이블 랜덤 액세스를 지목합니다.
5. 처방은 인덱스 키 순서 재정렬(CF 개선) 또는 커버링 인덱스입니다. 배열 크기·인덱스 재생성·버퍼 캐시는 291,000번의 랜덤 블록 방문을 없애지 못합니다.

■ 한 줄 정리 ArraySize 1,000으로 왕복 가설을 기각하고 CF 밀도 0.97($291,000 / 300,000$, 최댓값 1에 근접)로 테이블 랜덤 액세스를 지목해야 하며, 배열 크기·인덱스·버퍼 캐시는 병목을 잘못 짚은 처방이다.

문제 8. 정답 ④

인덱스 전체를 읽는 두 방식은 읽는 순서·I/O 단위·정렬 보장이 정확히 반대입니다. 이름이 비슷해 헷갈리지만 성질은 대칭입니다.

	읽는 순서	I/O 단위	키 순서 정렬	병렬
Index Full Scan	리프 논리적 연결순서	Single Block	보장 0	보통 X
Index Fast Full	세그먼트 물리적 순서	Multiblock	보장 X	가능

정렬이 필요한 쿼리 → Index Full Scan (키 순서 유지 → ORDER BY 대체)
 대량을 빨리 훑기 → Index Fast Full Scan (Multiblock·병렬, 순서는 깨짐)

④는 이 두 방식의 성질을 통째로 맞바꿔 놓았습니다. 리프의 논리적 연결 순서를 Single Block으로 따라가며 키 순서를 유지하는 것은 Index Full Scan이고, 세그먼트를 물리적 순서대로 Multiblock으로 읽어 정렬이 깨지는 것은 Index Fast Full Scan입니다. ④는 이 둘의 설명을 서로 바꿔 붙였으므로 정반대 서술입니다.

①·②·③은 옳습니다. 커버링으로 테이블 Random 액세스를 없애는 원리(①), 선두 컬럼 조건이 없어도 커버링이면 Index Full Scan으로 처리 가능한 점(②), Index Fast Full Scan의 물리적·Multiblock·병렬 성질과 정렬 미보장(③)이 모두 두

스캔의 실제 동작에 부합합니다.

선택지	판정	이유
①	○	참조 컬럼이 인덱스에 다 들어 있으면 테이블을 되짚지 않고 인덱스만 읽어, ROWID 기반 테이블 Random 액세스를 제거합니다
②	○	선두 컬럼 조건이 없어 Range Scan을 못 써도, 필요한 컬럼이 인덱스에 있으면 Index Full Scan으로 테이블 액세스 없이 처리할 수 있습니다
③	○	Index Fast Full Scan은 세그먼트를 물리적 순서로 Multiblock·병렬로 읽어 대량 처리에 유리하되, 결과가 키 순서로 정렬되지는 않습니다
④	×	논리적 연결 순서·Single Block·키 순서 정렬은 Index Full Scan의 성질이고, 물리적 순서·Multiblock·정렬 미보장은 Index Fast Full Scan의 성질입니다. 둘의 설명을 서로 맞바꾼 정반대 서술입니다

■ 이 문제의 핵심

1. Index Full Scan = 리프 논리적 순서·Single Block·키 순서 정렬(ORDER BY 대체 가능).
2. Index Fast Full Scan = 세그먼트 물리적 순서·Multiblock·병렬, 대신 정렬은 깨짐.
3. 두 방식의 읽는 순서·I/O 단위·정렬 보장은 서로 반대입니다. 맞바꾸면 오답입니다.
4. 커버링(인덱스 전용 스캔)은 참조 컬럼이 인덱스에 다 있을 때 테이블 Random 액세스를 제거합니다.

■ 한 줄 정리 리프 논리적 순서로 Single Block·정렬 유지하는 것은 Index Full Scan, 물리적 순서로 Multiblock·정렬이 깨지는 것은 Index Fast Full Scan인데, 두 설명을 서로 맞바꾼 ④는 정반대 서술입니다.

문제 9. 정답 ④

외래키 인덱스의 이득은 특정 상황의 동시성 문제(Lock 경합)를 푸는 것이지, 자식 테이블의 DML 자체를 가볍게 하는 것이 아닙니다. 두 축을 구분해야 합니다.

외래키 인덱스가 '완화'하는 것

= 부모 키 DELETE/UPDATE 시 자식에 걸리던 넓은 Lock + 자식 전체 스캔
(동시성/경합의 문제)

외래키 인덱스가 '더하는' 것

= 자식 테이블의 INSERT/UPDATE/DELETE마다 이 인덱스도 갱신
(DML 유지 부하 → 오히려 늘어남)

∴ '인덱스가 늘수록 DML이 가벼워진다'는 성립하지 않는다

④는 ①에서 말한 특정 시나리오의 Lock 완화라는 부분 이득을, "자식의 모든 DML 부하를 낮춘다 → 인덱스가 늘수록 DML이 가벼워진다"는 전체 명제로 부풀렸습니다. 외래키 인덱스가 줄여 주는 것은 부모 키 변경 시의 Lock·스캔이라는 동시성 부담이고, 정작 자식 테이블의 INSERT·UPDATE·DELETE에는 다른 인덱스와 똑같이 갱신 부하를 더합니다. 그래서 "인덱스가 늘수록 DML이 가벼워진다"는 결론은 DML 부하의 방향을 거꾸로 세운 과잉 일반화입니다.

①·②·③은 옳습니다. 인덱스 없는 외래키가 부모 키 변경 시 자식에 Lock·전체 스캔을 유발한다는 점(①), 외래키 인덱스가 그 부담을 덜고 조인 진입 경로로도 쓰인다는 점(②), 그럼에도 유지 부하를 더하므로 이득과 비용을 견주어야 한다는 점(③)이 모두 맞습니다.

선택지	판정	이유
①	○	외래키에 인덱스가 없으면 부모 키 DELETE/UPDATE 시 자식에 넓은 Lock을 걸고 전체 스캔하는 경향이 있어, 자식의 동시 DML 경합을 부릅니다
②	○	외래키 인덱스는 그 Lock·스캔 부담을 덜어 주고, 부모-자식 조인에서 자식 쪽 진입 경로로도 활용돼 조회에 도움이 됩니다
③	○	외래키 인덱스도 인덱스라 자식 DML마다 갱신 부하를 더하므로, 동시성·조회 이득과 유지 비용을 견주어 생성 여부를 판단해야 합니다
④	×	외래키 인덱스가 완화하는 것은 부모 키 변경 시의 Lock·스캔(동시성)일 뿐이며, 자식의 INSERT·UPDATE·DELETE에는 오히려 갱신 부하를 더합니다. "인덱스가 늘수록 DML이 가벼워진다"는 부분 이득을 전체로 부풀린 과잉 일반화입니다

■ 이 문제의 핵심

1. 인덱스 없는 외래키는 부모 키 DELETE/UPDATE 시 자식에 넓은 Lock·전체 스캔을 유발해 동시성 경합을 부릅니다.
2. 외래키 인덱스는 그 Lock·스캔(동시성)을 완화하고, 부모-자식 조인의 진입 경로로도 쓰입니다.

3. 외래키 인덱스도 자식 DML에는 갱신 부하를 더합니다. 완화되는 축(동시성)과 늘어나는 축(DML 유지)은 다릅니다.

4. 따라서 생성 여부는 동시성·조회 이득과 DML 유지 비용의 맞교환으로 판단합니다.

- 한 줄 정리 외래키 인덱스가 완화하는 것은 부모 키 변경 시의 Lock·스캔이라는 동시성 부담일 뿐 자식의 DML은 오히려 무거워지므로, "인덱스가 늘수록 DML이 가벼워진다"는 ④는 부분 이득을 전체로 부풀린 과잉 일반화입니다.

문제 10. 정답 ③

플랜을 세 가지 좌표 — 조인 방식, 빌드/프로브 순서, 액세스 방식 — 로 읽고, 힌트만 다른 네 SQL을 거꾸로 맞춥니다.

Id 2 HASH JOIN → 조인 방식은 해시 조인
 Id 3 TABLE ACCESS FULL 채용공고 ← HASH JOIN의 첫(위) 자식 = Build Input(선행)
 Id 4 TABLE ACCESS FULL 지원내역 ← HASH JOIN의 둘째(아래) 자식 = Probe Input(후행)

해시 조인에서 플랜의 위쪽 자식이 Build Input이고, 이는 조인 순서에서 선행 테이블입니다. 여기서는 채용공고(Id 3)가 Build, 지원내역(Id 4)이 Probe이고, 두 입력 모두 **TABLE ACCESS FULL**(인덱스 없이 전체 스캔)입니다. 이 세 가지를 동시에 만족하는 힌트는 선행=채용공고(c) + 해시 조인, 곧 **LEADING(c a) USE_HASH(a)**입니다.

- ③ LEADING(c a) USE_HASH(a) : 선행 채용공고=Build(위), 지원내역=Probe(아래), 해시 → Id 2~4와 일치
- ① LEADING(a c) USE_HASH(c) : 선행 지원내역=Build → 위 자식이 지원내역이어야 함 → 플랜은 채용공고와 위
- ② LEADING(c a) USE_NL(a) : NESTED LOOPS + 지원내역 인덱스 액세스 → 플랜은 HASH JOIN·FULL
- ④ LEADING(c a) USE_MERGE(a) : MERGE JOIN + 양쪽 SORT JOIN → 플랜에 SORT JOIN 없음

①은 빌드/프로브가 뒤바뀝니다. **LEADING(a c)**는 지원내역을 선행=Build로 지정하므로 HASH JOIN의 위 자식이 지원내역이어야 하는데, 플랜의 위 자식은 채용공고입니다.

②는 조인 방식이 다릅니다. **USE_NL(a)**는 NESTED LOOPS를 만들고 후행 지원내역을 인덱스로 탐색합니다. 플랜의 **HASH JOIN·양쪽 TABLE ACCESS FULL**과 어긋납니다.

④도 조인 방식이 다릅니다. **USE_MERGE(a)**는 소트 머지 조인이라 두 입력 위에 각각 **SORT JOIN** 노드가 생깁니다. 플랜엔 그 노드가 없습니다.

빌드 순서 하나(①), 조인 방식 둘(②·④)을 각각 플랜의 관측 가능한 노드로 배제하면, 이 플랜을 정확히 내는 것은 ③뿐입니다.

선택지	판정	이유
①	×	LEADING(a c) 가 지원내역을 선행=Build로 지정해 HASH JOIN의 위 자식이 지원내역이 됩니다. 플랜의 위 자식(Id 3)은 채용공고여서 빌드/프로브가 스왑된 SQL입니다
②	×	USE_NL(a) 는 NESTED LOOPS와 지원내역 인덱스 탐색을 만듭니다. 플랜은 Id 2 HASH JOIN에 양쪽이 TABLE ACCESS FULL이라 조인 방식이 다릅니다
③	○	LEADING(c a) USE_HASH(a) 가 선행 채용공고=Build(Id 3 위)·지원내역=Probe(Id 4 아래)·해시 조인·양쪽 Full Scan을 그대로 만들어 Id 2~4와 일치합니다
④	×	USE_MERGE(a) 는 소트 머지 조인이라 두 입력 위에 SORT JOIN 이 붙습니다. 플랜엔 SORT JOIN이 없고 SORT GROUP BY만 있어 조인 방식이 다릅니다

▪ 이 문제의 핵심

- 실행계획→SQL 역추론은 세 좌표를 읽습니다. 조인 방식(HASH JOIN), 빌드/프로브 순서(위 자식=Build=선행), 액세스 방식(양쪽 TABLE ACCESS FULL).
- 해시 조인의 위쪽 자식이 Build Input이자 선행 테이블입니다. 플랜은 채용공고(Id 3)가 Build, 지원내역(Id 4)이 Probe이므로 **LEADING(c a)**가 필요합니다.
- 조인 방식을 바꾸면 노드가 바뀝니다. **USE_NL**은 NESTED LOOPS+인덱스, **USE_MERGE**는 MERGE JOIN+SORT JOIN을 만들어 HASH JOIN 플랜과 어긋납니다.
- 선행/후행을 뒤집으면 위·아래 자식이 뒤바뀝니다. **LEADING(a c)**는 지원내역을 Build로 올려 플랜과 모순됩니다 — 방식·순서를 뒤바꾼 값스왑 오답입니다.

- 한 줄 정리 플랜이 HASH JOIN + 위 자식 채용공고(Build) + 아래 자식 지원내역(Probe) + 양쪽 Full Scan이므로, 이를 내는 SQL은 **LEADING(c a) USE_HASH(a)**인 ③뿐이고 나머지는 조인 방식·빌드 순서를 뒤바꾼 SQL이다.

문제 11. 정답 ④

EXISTS가 세미(semi) 조인으로 폴리드, **NOT EXISTS·NOT IN**은 '짝이 없는 행만 통과'시키는 안티(anti) 조인으로 풀립니다. 그런데 **NOT IN**과 **NOT EXISTS**는 NULL을 만나는 순간 의미가 갈립니다.

NOT IN (a, b, NULL) 의 내부 전개

$x \text{ NOT IN } (a, b, \text{NULL})$
 $= x \diamond a \text{ AND } x \diamond b \text{ AND } x \diamond \text{NULL}$
└── UNKNOWN (참도 거짓도 아님)
 → AND 사슬 전체가 TRUE가 될 수 없음 → 어떤 x도 통과 못 함 → 결과 없음

NOT EXISTS (상관 서브쿼리)

'짜이 되는 행이 있는가?'를 상관 조건으로 직접 확인
 → 서브쿼리에 NULL이 있어도 짜 없는 바깥 행은 정상 통과

즉 서브쿼리 쪽에 NULL이 섞일 수 있으면 **NOT IN**은 결과가 통째로 비어 버릴 수 있고, 이는 **NOT EXISTS**와 다른 결과입니다. 두 형태가 논리적으로 같지 않으므로, 옵티마이저는 **NOT IN**을 함부로 단순 안티 조인으로 바꾸지 못하고 NULL을 고려하는 별도 형태(null-aware anti join)로 다루거나, 컬럼이 **NOT NULL**임이 보장될 때만 일반 안티 조인으로 변환합니다. ④가 이 NULL 의미 차이를 정확히 진술합니다.

나머지는 '부정 = 같은 것'이라는 반사적 연상에 걸린 진술입니다.

- ①: **NOT IN**과 **NOT EXISTS**를 '같은 부정'으로 반사적으로 묶었지만, 위 전개대로 NULL이 끼면 결과가 달라집니다. 서로 바꿔 쓸 수 있는 것은 서브쿼리 컬럼과 바깥 컬럼이 모두 NULL을 갖지 않을 때뿐입니다.
- ②: NULL로 결과가 비는 쪽은 **NOT EXISTS**가 아니라 **NOT IN**입니다. 두 형태의 NULL 반응을 서로 뒤바꿔 놓았습니다.
- ③: 서브쿼리 컬럼이 **NOT NULL**이면 $x \diamond \text{NULL}$ 문제가 사라져 **NOT IN**은 안전하게 안티 조인으로 변환됩니다. 등치(=) 부정이면 **HASH JOIN ANTI**로도 풀리며, 단순 필터로만 처리된다는 단정은 틀렸습니다.

선택지	판정	이유
①	×	NULL이 섞이면 NOT IN 은 결과가 비어 NOT EXISTS 와 달라집니다. '부정은 다 같다'는 반사적 연상이며, 두 형태는 NULL이 없을 때만 교체 가능합니다
②	×	결과가 비는 쪽은 NOT EXISTS 가 아니라 NOT IN 입니다. 두 형태의 NULL 반응을 서로 뒤바꾼 진술입니다
③	×	컬럼이 NOT NULL 이면 NOT IN 은 안전하게 안티 조인으로 변환되며, 등치 부정은 HASH JOIN ANTI 로도 풀립니다. 단순 필터 전용이 아닙니다
④	○	$x \diamond \text{NULL}$ 이 UNKNOWN이라 NOT IN 은 NULL이 있으면 결과가 비어 NOT EXISTS 와 달라지고, 옵티마이저도 두 형태를 같은 안티 조인으로 취급하지 않습니다

▪ 이 문제의 핵심

- NOT EXISTS**·**NOT IN**은 안티 조인으로 풀린다. 세미 조인이 '짜 있는 행'을 통과시키는 것과 대칭으로, 안티 조인은 '짜 없는 행'만 통과시킵니다.
- NULL에서 두 형태가 같린다. $x \diamond \text{NULL}$ 이 UNKNOWN이라, 서브쿼리에 NULL이 있으면 **NOT IN**은 결과가 통째로 비고 **NOT EXISTS**는 정상 동작합니다.
- NOT NULL** 보장이 있어야 안전하게 변환된다. 컬럼이 **NOT NULL**이면 **NOT IN**도 안티 조인(등치면 **HASH JOIN ANTI**)으로 풀리고, 아니면 null-aware 형태로 처리됩니다.
- '부정은 다 같다'는 반사적 연상이다. 두 형태를 무조건 교체하거나 NULL 반응을 뒤바꾸면 결과가 달라집니다.

▪ 한 줄 정리 **NOT IN**은 서브쿼리에 NULL이 있으면 $x \diamond \text{NULL}$ 이 UNKNOWN이 되어 결과가 통째로 비므로 **NOT EXISTS**와 결과가 같리고, 옵티마이저도 이 NULL 의미 차이 때문에 두 형태를 같은 안티 조인으로 취급하지 않습니다.

문제 12. 정답 ④

이 세 변환은 하는 일도 발동 조건도 서로 독립입니다. 이름·형태의 인상으로 순서 종속을 만들어 붙이면 틀립니다.

서브쿼리 Unnesting : 중첩 서브쿼리를 조인으로 평탄화 (조인 방식·순서를 비용으로 선택)
 OR-Expansion : OR 조건을 UNION ALL 브랜치로 분리 (브랜치별 인덱스 액세스)
 조건절 이행 : 등치 연쇄로 새 조건 유도 ($A=B \wedge B=10 \rightarrow A=10$, 새 액세스 경로)
 ↑ 세 변환은 각자 적용 여부를 비용으로 판단하는 독립 변환

④는 "둘 다 쿼리를 여러 조각으로 나눈다"는 표면 인상에서 "그러니 Unnesting은 OR-Expansion을 거친 뒤에야 적용된다"는 순서 종속을 끌어냈습니다. 이는 반사적 연상입니다. 서브쿼리에 **OR**가 있어도 옵티마이저는 Unnesting과 OR-Expansion을 각각 독립적으로 판단할 뿐이며, 한쪽이 다른 쪽의 선행 단계가 되는 강제된 순서는 없습니다. 그래서 ④가 부적절한 진술입니다.

나머지는 모두 옳습니다.

①: Unnesting이 불가능하거나 이득이 없으면 서브쿼리는 조인으로 풀리지 않고 필터 서브쿼리(FILTER)로 남아 바깥 행마다 평가됩니다.

②: OR-Expansion은 **OR** 조건을 **UNION ALL** 브랜치로 쪼개, 브랜치마다 자기에 맞는 인덱스를 액세스 조건으로 쓰게 합니다.

③: 조건절 이행이 유도한 새 조건은 원래 없던 인덱스 경로를 열 수 있고, 옵티마이저는 그 경로를 후보에 넣어 비용으로 원래 경로와 견뎌줍니다.

선택지	판정	이유
①	○	Unnesting이 불가능·무익하면 서브쿼리는 조인으로 풀리지 않고 FILTER 오퍼레이션으로 남아 바깥 행마다 평가됩니다
②	○	OR-Expansion은 OR 조건을 UNION ALL 브랜치로 분리해 브랜치마다 다른 인덱스를 액세스 조건으로 쓰게 합니다
③	○	조건절 이행이 유도한 새 조건은 없던 인덱스 경로를 열어, 옵티마이저가 후보에 넣고 비용으로 원래 경로와 비교합니다
④	×	Unnesting과 OR-Expansion은 독립 변환입니다. OR 가 있다고 Unnesting이 OR-Expansion을 선행 단계로 거치는 강제 순서는 없으며, '둘 다 나눈다'에서 순서 종속을 끌어낸 반사적 연상입니다

■ 이 문제의 핵심

1. Unnesting·OR-Expansion·조건절 이행은 독립 변환. 각자 적용 여부를 비용으로 판단하며, 한쪽이 다른 쪽의 선행 단계가 아닙니다.
2. Unnesting이 막히면 FILTER로 잔존. 조인으로 못 풀거나 이득이 없으면 서브쿼리는 필터 오퍼레이션으로 남습니다.
3. OR-Expansion = OR를 UNION ALL 브랜치로. 브랜치마다 다른 인덱스를 액세스 조건으로 쓸 수 있습니다.
4. 조건절 이행 = 새 조건 유도 → 새 액세스 경로. 유도된 경로는 후보로 들어가 비용으로 원래 경로와 비교됩니다.

■ 한 줄 정리 Unnesting·OR-Expansion·조건절 이행은 각자 비용으로 판단되는 독립 변환이므로, "둘 다 쿼리를 나누니 Unnesting은 OR-Expansion을 거친 뒤에야 적용된다"는 진술은 표면 인상에서 순서 종속을 끌어낸 반사적 연상입니다.

문제 13. 정답 ③

편중 칼럼에 바인드 변수를 쓰면 두 힘이 부딪칩니다. 커서 공유는 텍스트가 같으니 계획을 재사용하려 하고, 히스토그램은 값마다 최적 계획이 다르다고 말합니다. 오라클은 이 충돌을 Bind Peeking과 Adaptive Cursor Sharing으로 처리하는데, 그 흔적이 v\$sql 자식 커서에 그대로 남습니다.

[자식 커서 판독]

```
child 0 : SENSITIVE=Y AWARE=N PLAN 2716355091(FULL) EXEC 40
        → 첫 peek 계획이 굳어 값과 무관하게 재사용된 흔적
child 1 : SENSITIVE=Y AWARE=Y PLAN 3182904677(INDEX) EXEC 7 ← 회소값용
child 2 : SENSITIVE=Y AWARE=Y PLAN 2716355091(FULL) EXEC 18 ← 다수값용
        → ACS가 값 구간별로 서로 다른 계획을 분기함 (PLAN 두 종류)
```

③은 "바인드 변수를 쓰면 분포와 무관하게 항상 단일 최적 계획이 나와 어떤 값이든 똑같이 효율적"이라고 단정합니다. 이것이 정규화 가정의 오류입니다. 커서 텍스트가 하나로 정규화(공유)된다는 사실을, "계획도 하나로 최적화돼 있다"로 착각한 것입니다. 실제로는 편중 칼럼이라 '연체'(0.1%)엔 인덱스 스캔이, '번역완료'(97%)엔 풀 스캔이 최적이라 최적 계획이 값에 따라 갈립니다. 만약 ③처럼 항상 단일 최적 계획이라면 자식 커서의 PLAN_HASH_VALUE가 하나여야 하는데, 요약표엔 2716355091(FULL)과 3182904677(INDEX) 두 종류가 있습니다. 텍스트 공유(커서 공유)와 계획 최적화는 별개 축이며, 그래서 오라클은 오히려 bind-aware 자식을 나눠 계획을 분기합니다. 부적절한 진술은 ③입니다.

①②④는 히스토그램→bind-sensitive(①), 첫 peek 계획이 굳는 위험(②), ACS의 bind-aware 분기(④)를 요약표 그대로 옮겨 서술합니다.

선택지	판정	이유
①	○	히스토그램이 편중을 알려주어 바인드 실행이 bind-sensitive(자식 0 SENSITIVE=Y)가 됩니다. 첫 바인드 값을 peek해 계획을 세우는 동작 그대로입니다
②	○	첫 peek 계획이 커서에 굳으면 이후 다른 값에도 재사용됩니다. 자식 0이 EXEC 40으로 한 계획을 반복한 것이 그 위험을 보여줍니다
③	×	커서 텍스트가 하나로 공유돼도 계획이 하나로 최적화되는 것은 아닙니다. 요약표엔 PLAN이 FULL·INDEX 두 종류라 "항상 단일 최적"은 성립하지 않습니다
④	○	ACS가 bind-sensitive→bind-aware로 전환해 값 구간별 자식 커서를 만듭니다. 자식 1·2가 서로 다른 PLAN_HASH_VALUE로 갈린 것이 근거입니다

▪ 이 문제의 핵심

1. 커서 공유(텍스트 정규화)와 계획 최적화는 별개 축입니다. 바인드로 텍스트가 하나가 돼도 최적 계획까지 하나가 되는 것은 아닙니다.
2. 히스토그램 + 바인드 = Bind Peeking. 파싱 시점 첫 바인드 값을 엿보아 계획을 세우고, 그 계획이 커서에 굳어 다른 값에 재사용될 수 있습니다(첫 peek 굳음).
3. Adaptive Cursor Sharing은 편중 칼럼에서 커서를 bind-sensitive→bind-aware로 전환해 값 구간별 별도 지식 커서(다른 계획)를 만듭니다 — PLAN_HASH_VALUE가 갈리는 것이 그 증거입니다.
4. "바인드면 항상 단일 최적 계획"은 텍스트 공유를 계획 공유로 넘겨짚은 정규화 가정의 착각입니다.
 - 한 줄 정리 편중 칼럼에 바인드를 쓰면 텍스트는 하나로 공유돼도 최적 계획은 값마다 달라 PLAN이 여러 종류로 갈리므로, "바인드면 항상 단일 최적 계획"은 커서 공유를 계획 최적화로 착각한 진술입니다.

문제 14. 정답 ③

히스토그램은 컬럼 값의 분포를 담은 통계입니다. 담은 방식이 두 가지로 갈립니다.

값의 종류(NDV) ≤ 버킷 수 → 도수분포(frequency) : 값마다 버킷 1개, 빈도를 그대로 저장
 값의 종류(NDV) > 버킷 수 → 높이균형(height-balanced) : 여러 값을 한 버킷에 묶어 분포 근사
 히스토그램 없음 → 균등(uniform) 분포 가정 : 모든 값이 고르게 나온다고 간주

히스토그램이 없으면 옵티마이저는 값이 고르게 분포한다고 가정해 등치 선택도를 대략 $1/NDV$ 로 잡습니다. 실제 분포가 한쪽으로 치우친 컬럼에서는 이 가정이 빗나가, 흔한 값은 카디널리티를 과소평가하고 드문 값은 과대평가하게 됩니다. 히스토그램은 도수분포든 높이균형이든 이 균등 가정을 실제 분포로 대체해 편중된 값의 선택도를 정밀하게 만듭니다. ③이 두 히스토그램의 생성 기준과 그 효과를 정확히 진술합니다.

나머지는 서로 독립인 두 축을 인과로 엮은 별개 축 혼동입니다.

- ①: NDV(값의 다양성)와 분포의 균등성(편중 여부)은 별개 축입니다. NDV가 커도 특정 값에 행이 몰릴 수 있고, NDV가 작아도 심하게 치우칠 수 있습니다. 히스토그램의 실익은 NDV 크기가 아니라 분포가 치우쳤는가로 갈립니다.
- ②: 추정 정확도와 실행 비용·속도는 별개 축입니다. 히스토그램으로 선택도가 정확해지면 비용 추정이 실제에 가까워질 뿐, 그 값이 흔한 값이면 오히려 비용이 더 높게(그리고 더 맞게) 잡힐 수도 있습니다. '정확 = 저비용 = 빠름'이 아닙니다.
- ④: 버킷 수(히스토그램 해상도)와 NDV(실제 값의 개수)는 별개 축입니다. 버킷을 촘촘히 잡아도 컬럼이 가진 서로 다른 값의 수 자체는 변하지 않습니다. 버킷은 분포를 얼마나 잘게 근사하느냐일 뿐, NDV를 늘리지 않습니다.

선택지	판정	이유
①	×	NDV(다양성)와 분포 균등성(편중)은 독립 축입니다. NDV가 커도 값이 몰릴 수 있어, 히스토그램 실익은 NDV 크기가 아니라 편중 여부로 갈립니다
②	×	추정 정확도와 실행 비용·속도는 별개 축입니다. 선택도가 정확해지면 비용이 실제에 가까워질 뿐, 흔한 값이면 비용이 더 높게 잡힐 수도 있어 '정확=빠름'이 아닙니다
③	○	NDV와 버킷 수의 대소로 도수분포·높이균형이 갈리며, 어느 쪽이든 히스토그램이 없을 때의 균등 가정보다 편중 값 선택도를 정밀하게 추정합니다
④	×	버킷 수(해상도)와 NDV(실제 값 개수)는 독립 축입니다. 버킷을 촘촘히 잡아도 컬럼의 서로 다른 값 수는 변하지 않아 NDV가 늘지 않습니다

▪ 이 문제의 핵심

1. 도수분포 vs 높이균형은 NDV와 버킷 수의 대소로 갈린다. 값 종류가 버킷보다 적으면 도수분포, 많으면 높이균형입니다.
2. 히스토그램이 없으면 균등 분포 가정(선택도 $\approx 1/NDV$). 편중된 컬럼에서 이 가정이 빗나가므로 히스토그램으로 보정합니다.
3. NDV 크기 ≠ 분포 균등성. 히스토그램의 실익은 값이 얼마나 치우쳤는가로 결정되지, NDV가 크고 작음으로 결정되지 않습니다.
4. 추정 정확도 ≠ 실행 속도, 버킷 수 ≠ NDV. 정확한 추정이 곧 저비용은 아니고, 버킷을 늘려도 실제 값의 개수는 변하지 않습니다.
 - 한 줄 정리 도수분포·높이균형 히스토그램은 NDV와 버킷 수의 대소로 갈리며 균등 분포 가정을 실제 분포로 대체해 편중 값 선택도를 정밀하게 만드는 것이 옳고, NDV 크기를 분포 균등성으로·추정 정확도를 실행 속도로·버킷 수를 NDV로 잇는 진술은 모두 독립인 두 축을 인과로 엮은 혼동입니다.

문제 15. 정답 ①

이 배치는 대량 적재의 3대 무기 — Direct Path Insert(APPEND) · 병렬 DML · minimal logging — 를 EXCHANGE PARTITION과 결합한 정석입니다. 각 무기의 작동 경계를 정확히 짚어야 옳은 진술을 가릴 수 있습니다.

[각 단계의 실제 동작]

INSERT /*+ APPEND */ → HWM 위 새 블록에 직접 기록(버퍼캐시 우회, Direct Path)
 + 대상 NOLOGGING + DB가 FORCE LOGGING 아님 → 데이터 리두 최소화(minimal logging)
 PARALLEL(t,4) → SELECT뿐 아니라 INSERT까지 병렬로 하려면
 ENABLE PARALLEL DML 선행 필수 (없으면 INSERT는 직렬)
 EXCHANGE PARTITION → 세그먼트 포인터만 맞추는 덱서너리 연산 → 데이터 이동 없음

①은 세 가지를 모두 정확히 서술합니다. APPEND는 HWM 위 직접 기록(Direct Path)이고, NOLOGGING + 비FORCE LOGGING 조건이 갖춰지면 데이터 리두가 minimal logging으로 줄며, EXCHANGE PARTITION은 세그먼트를 맞추는 메타데이터 연산이라 800만이든 8억이든 데이터를 옮기지 않아 즉시 끝납니다. 경계 조건(NOLOGGING이되 FORCE LOGGING이 아닐 것)까지 발문과 일치하므로 옳은 진술은 ①입니다.

②③④는 각 기능의 동작 경계를 뭉개 경계 오해입니다(아래 표).

선택지	판정	이유
①	○	APPEND는 HWM 위 Direct Path 기록이고, NOLOGGING+비FORCE LOGGING이면 데이터 리두가 최소화되며, EXCHANGE는 세그먼트를 맞추는 메타데이터 연산이라 즉시 끝납니다
②	×	힌트만으로는 SELECT만 병렬이 되고 INSERT는 직렬입니다. INSERT까지 병렬 DML로 돌리려면 ENABLE PARALLEL DML 선언이 반드시 선행돼야 합니다 — 이 문장은 필수입니다
③	×	Direct Path 적재는 대상 테이블에 배타적 테이블 잠금을 걸어, 커밋 전까지 다른 세션의 일반 DML을 막습니다. 자유로운 동시 DML은 불가능합니다
④	×	WITHOUT VALIDATION 은 범위 검증을 생략할 뿐 자동 재배포가 아닙니다. 범위를 벗어난 행이 섞이면 그대로 파티션에 들어가 프루닝·조화가 어긋납니다

▪ 이 문제의 핵심

1. Direct Path Insert(APPEND) 는 HWM 위 새 블록에 직접 기록하며, 그동안 대상 테이블에 배타적 잠금을 걸어 동시 DML을 막습니다.
2. minimal logging의 경계: 객체가 NOLOGGING이고 DB가 FORCE LOGGING이 아니며 Direct Path일 때만 데이터 리두가 최소화됩니다 — 세 조건이 함께여야 합니다.
3. 병렬 DML의 경계: **PARALLEL** 힌트만으로는 SELECT만 병렬입니다. INSERT까지 병렬로 하려면 **ENABLE PARALLEL DML**을 먼저 켜야 합니다.
4. EXCHANGE PARTITION은 세그먼트를 맞추는 메타데이터 연산이라 데이터를 옮기지 않습니다. 단 **WITHOUT VALIDATION**은 검증을 생략할 뿐 잘못된 범위 행을 교정해 주지 않습니다.

▪ 한 줄 정리 APPEND Direct Path + NOLOGGING(비FORCE LOGGING) + 세그먼트를 맞추는 EXCHANGE PARTITION의 경계를 모두 옳게 짚은 ①이 적절하며, 병렬 DML 활성화·Direct Path 잠금·WITHOUT VALIDATION 검증은 각각 경계를 뭉개 오답입니다.

문제 16. 정답 ④

복합 파티션에서 프루닝은 두 축을 따로 판정합니다. RANGE 축(접수일자)은 경계값과 대조해, HASH 축(배출자번호)은 정확한 값의 해시 결과로 서브파티션을 좁힙니다. 두 축 모두 키에 가공이 없어야 대조·해시가 가능합니다.

[조희별 접근 범위]

- ① 접수일자 ≥ 2026-07-01 → RANGE: p_2026h2 · p_max / HASH: 서브 4개 전부(회원 조건 없음)
 ② 배출자번호 = 8801234 → RANGE: 전 파티션 / HASH: 각 파티션 내 서브 1개
 ③ 접수일자=2026-08-10 & 회원=8801234 → RANGE: p_2026h2 1개 / HASH: 서브 1개 → 최종 1개
 ④ SUBSTR(TO_CHAR(배출자번호),1,3)='880' → 키 가공 → 해시 대조 불가 → 서브 전부 스캔

해시 파티셔닝은 키 값 전체를 해시 함수에 넣어 서브파티션 번호를 계산합니다. 그래서 서브파티션 프루닝은 **배출자번호 = 8801234**처럼 등치 조건(정확한 값) 일 때만 걸립니다. ④는 **SUBSTR(TO_CHAR(배출자번호),1,3)='880'**로 키를 가공했습니다. **8801234**와 **8809999**는 앞 3자리가 같아도 해시 결과가 전혀 다른 서브파티션으로 흩어지므로, 옵티마이저는 "앞 3자리 880"을 특정 서브파티션에 대응시킬 수 없습니다. 결국 모든 서브파티션을 스캔한 뒤 각 행에서 SUBSTR을 평가해 필터합니다. "배출자번호 앞자리로 겨냥한다"는 의미상의 부분적 진실이 있어도, 키 가공 한 줄이 최선(서브 1개)을 최악(서브 전부)으로 뒤집습니다. 그래서 설명대로 프루닝이 작동한다고 볼 수 없는 것은 ④입니다.

①②③은 RANGE 단독 프루닝(①), HASH 등치 서브파티션 프루닝(②), 두 축 결합 단일 서브파티션(③)을 모두 옳게 서술합니다.

선택지	판정	이유
①	○	접수일자 범위로 RANGE 프루닝이 걸려 p_2026h2·p_max만 접근하고, 회원 조건이 없어 그 안 서브파티션 4개는 전부 읽습니다
②	○	배출자번호 등치 조건이라 RANGE는 못 좁혀도 각 파티션 안에서 해시 매핑된 서브파티션 1개씩만 접근하는 서브파티션 프루닝이 작동합니다
③	○	접수일자 등치로 p_2026h2 1개, 배출자번호 등치로 그 안 서브 1개까지 좁혀 최종 단일 서브파티션만 읽습니다
④	×	SUBSTR(TO_CHAR(배출자번호)) 로 키를 가공하면 해시 대조가 불가능해 전 서브파티션을 스캔합니다. "앞 3자리로 겨냥"은 프루닝을 보장하지 못하는 부분진실입니다

▪ 이 문제의 핵심

1. 복합 파티션의 프루닝은 RANGE 축·HASH 축을 따로 판정합니다. 각 축의 키가 가공 없이 걸려야 대조·해시가 가능합니다.
2. HASH 서브파티션 프루닝은 등치(정확한 값) 조건에서만 걸립니다 — 해시 함수는 값 전체를 넣어 버킷을 계산하기 때문입니다.
3. 두 축을 모두 등치로 걸면(③) 파티션 1개 × 서브파티션 1개로 최종 단일 세그먼트까지 좁혀집니다.
4. 키를 가공(SUBSTR·TO_CHAR·범위) 하면 해시 대조가 불가능해 서브파티션을 전부 스캔합니다 — "앞자리로 겨냥"이라는 부분진실이 함정입니다.

▪ 한 줄 정리 HASH 서브파티션 프루닝은 배출자번호 등치 조건에서만 걸리며, **SUBSTR(TO_CHAR(배출자번호))** 같은 키 가공은 앞자리가 같아도 해시가 흩어져 전 서브파티션을 스캔하므로 ④가 부적절합니다.

문제 17. 정답 ②

분배 방식은 조인 입력이 **PX SEND**를 거치는지, 그리고 조인을 감싸는 노드가 무엇인지로 읽습니다.

```

Id 4  PX PARTITION HASH ALL (Pstart 1~Pstop 64)   조인 전체를 파티션 단위로 감쌈
Id 5  HASH JOIN (Q1,00)                          파티션 집합 내부에서 조인
Id 6  HASH JOIN (Q1,00)                          파티션 집합 내부에서 조인
Id 7  TABLE ACCESS FULL 사용량집계 (Q1,00, 1~64)  조인 입력 - PX SEND 없음
Id 8  TABLE ACCESS FULL 공급계약 (Q1,00, 1~64)  조인 입력 - PX SEND 없음
Id 9  TABLE ACCESS FULL 단가적용 (Q1,00, 1~64)  조인 입력 - PX SEND 없음

```

세 조인 입력(Id 7·8·9)은 같은 TQ(Q1,00) 아래에서, 같은 **PX PARTITION HASH ALL**(Id 4) 안에서 읽힙니다. 그 사이 어디에도 **PX SEND/PX RECEIVE**가 없습니다. 세 테이블이 계약번호로 동일하게 64개 해시 파티셔닝돼 있으므로, 각 병렬 서버가 대응하는 파티션 집합(사용량집계 p_k ↔ 공급계약 p_k ↔ 단가적용 p_k)만 맞들어 조인합니다. 이것이 재분배가 사라지는 (full) 파티션 와이즈 조인입니다.

플랜에 유일하게 있는 **PX SEND**는 최상단 Id 2 **PX SEND QC (RANDOM)** — 결과를 QC로 넘기는 마지막 단계뿐입니다. 조인 입력 사이의 재분배 SEND는 하나도 없습니다.

Id 3 **HASH GROUP BY**도 집계 키가 파티션 키(계약번호)와 같습니다. 한 계약번호의 행은 한 파티션(=한 서버)에만 있으므로, 집계 역시 서버 로컬로 끝나 별도의 재분배 TQ가 필요 없습니다. 그래서 이 플랜엔 TQ가 Q1,00 하나뿐입니다.

```

② full PWJ   : 세 입력 모두 SEND 없음(같은 Id 4 PX PARTITION HASH ALL 아래) → Id 7·8·9와 일치
① partial PWJ : 한쪽은 로컬, 다른 쪽에 PX SEND PARTITION (KEY) → 어느 입력에도 SEND 없음
③ hash-hash   : 세 입력 위에 각각 PX SEND HASH → 조인 입력에 SEND 없음
④ broadcast   : 소형 쪽에 PX SEND BROADCAST → BROADCAST 노드 자체가 없음

```

조인 입력에 **PX SEND**가 전혀 없다는 사실이 partial PWJ·hash-hash·broadcast를 한꺼번에 배제합니다. 따라서 ②가 옳습니다.

선택지	판정	이유
①	×	partial 파티션 와이즈면 재분배되는 쪽에 PX SEND PARTITION (KEY) 가 있어야 하는데, 공급계약·단가적용(Id 8·9) 위에 SEND가 없습니다. 세 테이블이 이미 같은 키로 파티셔닝돼 있어 한쪽만 재분배할 이유도 없습니다
②	○	조인 입력 Id 7·8·9가 같은 Q1,00·같은 PX PARTITION HASH ALL(Id 4) 아래에서 SEND 없이 읽혀 대응 파티션 집합만 조인하고, Id 3 HASH GROUP BY도 파티션 키(계약번호) 기준이라 로컬로 끝나는 full 파티션 와이즈 조인입니다
③	×	hash-hash면 조인 입력 세 곳 위에 각각 PX SEND HASH 가 있어야 합니다. Id 7·8·9엔 SEND가 없고, Id 3은 SEND HASH가 아니라 파티션 로컬 HASH GROUP BY입니다
④	×	broadcast면 소형 쪽에 PX SEND BROADCAST 가 있어야 하는데 플랜에 BROADCAST 노드가 없습니다. 세 입력 모두 파티션 단위 로컬 스캔이라 복제 자체가 일어나지 않습니다

■ 이 문제의 핵심

1. full 파티션 와이즈 조인은 재분배가 전혀 없습니다. 세 테이블이 조인 키로 동일하게 파티셔닝돼 있어, 각 서버가 대응 파티션 집합만 로컬로 조인합니다.
2. 재분배 여부는 조인 입력의 **PX SEND** 유무로 판별합니다. Id 7·8·9 위에 SEND가 없고 모두 같은 PX PARTITION HASH ALL(Id 4) 아래에 있습니다.
3. 집계용 재분배도 여기서 없습니다. Id 3 HASH GROUP BY의 키가 파티션 키(계약번호)와 같아 서버 로컬로 끝나므로, TQ가 Q1,00 하나뿐이고 조인 위에 SEND HASH가 붙지 않습니다.
4. partial PWJ·hash-hash·broadcast는 조인 입력에 **PX SEND PARTITION (KEY)/PX SEND HASH/PX SEND BROADCAST**가 있어야 성립합니다. 이 플랜의 조인 입력엔 어느 SEND도 없습니다 — 분배 방식을 뒤바꾼 값스왑 오답입니다.

- 한 줄 정리 사용량집계·공급계약·단가적용이 계약번호로 동일하게 64 해시 파티셔닝돼 조인 입력 Id 7·8·9가 같은 PX PARTITION HASH ALL 아래에서 SEND 없이 대응 파티션 집합만 조인하고 집계 키까지 파티션 키와 같으므로, 재분배가 없는 full 파티션 와이즈 조인이다.

문제 18. 정답 ③

인덱스가 그룹핑의 소트를 지우려면 인덱스가 그룹 키의 정렬 순서를 그대로 제공해야 합니다. 단지 '인덱스가 있다'는 것만으로는 부족합니다.

SORT GROUP BY 생략 조건

- GROUP BY 컬럼이 인덱스 선두 컬럼의 정렬 순서와 일치해야 함
- 그래야 인덱스를 순서대로 훑으며 그룹 경계를 나눌 수 있음(NOSORT)
- '인덱스 존재'만으로는 불충분 (정렬 순서 일치가 핵심)

HASH GROUP BY

- 정렬이 아니라 해시 매칭으로 그룹을 모음
- 인덱스의 '정렬 순서'로 대체될 성질의 오퍼레이션이 아님

③은 이 부분 조건(정렬 순서 일치 + SORT GROUP BY에 한정)을 '인덱스가 있지만 하면 모든 집계 오퍼레이션이 사라진다'는 전체 등식으로 부풀린 하위항목 전체화입니다. 인덱스가 그룹 키 정렬 순서와 어긋나면 SORT GROUP BY는 생략되지 않고, HASH GROUP BY는 정렬을 쓰지 않으니 인덱스 정렬 순서로 지워지는 대상 자체가 아닙니다. 그래서 ③이 부적절한 진술입니다.

①·②·④는 옳습니다. SORT GROUP BY만 결과가 그룹 키 순서를 띠고 HASH GROUP BY는 그렇지 않다는 정렬 특성(①), 두 방식 모두 PGA work area에서 처리되고 초과 시 Temp 디스크로 저하된다는 처리 위치(②), 인덱스 정렬 대체는 SORT GROUP BY에만 걸리고 HASH GROUP BY는 대상이 아니라는 경계(④)가 모두 표준 설명입니다.

선택지	판정	이유
①	○	SORT GROUP BY는 정렬하며 집계해 결과가 그룹 키 순서를 띠지만, HASH GROUP BY는 해시 누적이라 결과가 그룹 키 순서로 정렬되어 나오지 않습니다
②	○	정렬·해시 그룹핑 모두 PGA work area에서 수행되고, 초과분은 Temp를 쓰는 디스크 처리로 넘어가 I/O 부하가 커집니다
③	×	SORT GROUP BY 생략은 인덱스가 그룹 키 정렬 순서와 일치할 때만이고, HASH GROUP BY는 정렬을 안 써 대상이 아닙니다. '인덱스가 있지만 하면 다 사라진다'는 부분 조건을 전체로 부풀린 하위항목 전체화입니다
④	○	인덱스 정렬 순서로 지워지는 것은 정렬 기반 SORT GROUP BY이며, 해시 매칭으로 그룹을 모으는 HASH GROUP BY는 정렬을 쓰지 않아 대체 대상이 아닙니다

▪ 이 문제의 핵심

1. GROUP BY는 SORT GROUP BY·HASH GROUP BY 두 방식으로 처리된다. 전자는 결과가 그룹 키 순서를 띠고, 후자는 정렬되어 나오지 않습니다.
2. 인덱스로 지워지는 건 정렬 순서가 맞는 SORT GROUP BY뿐. '인덱스 존재'만으로는 부족하고, 그룹 키가 인덱스 선두 정렬 순서와 일치해야 합니다.
3. HASH GROUP BY는 인덱스 정렬 대체 대상이 아니다. 정렬이 아니라 해시 매칭으로 그룹을 모으기 때문입니다.
4. 두 방식 모두 PGA work area에서 처리되며, 초과 시 Temp 디스크 처리로 저하됩니다.

▪ 한 줄 정리 인덱스로 지워지는 소트는 그룹 키 정렬 순서가 맞는 SORT GROUP BY에 한정되고 HASH GROUP BY는 정렬을 쓰지 않아 대상이 아닌데, "인덱스가 있지만 어떤 집계 오퍼레이션이든 사라진다"는 진술은 부분 조건을 전체 등식으로 부풀린 하위항목 전체화입니다.

문제 19. 정답 ①

오라클은 두 격리수준 모두 MVCC(다중 버전)로 읽어 쓰기를 기다리지 않지만, 읽기 일관성의 기준 시점(스냅샷 SCN) 이 다릅니다. READ COMMITTED는 문장 수준 — 매 SELECT가 그 문장 시작 시점의 커밋 값을 봅니다. SERIALIZABLE은 트랜잭션 수준 — 트랜잭션 시작 시점의 커밋 값을 트랜잭션 내내 고정해 봅니다. Undo 기반 CR(Consistent Read) 블록으로 과거 버전을 재구성한다는 원리는 같고, 되감을 시점만 다릅니다.

[두 번째 읽기(t3) 값 계산]

공통: t1 첫 읽기 = 30000 (아직 TX1 미커밋)
t2에 TX1이 30000 → 35000 적립·COMMIT

READ COMMITTED (문장 수준)

t3 SELECT는 't3 시점'의 커밋 값을 새로 읽음 → TX1 커밋(35000) 반영
→ 두 번째 읽기 = 35000 (첫 읽기 30000과 달라짐 = Non-Repeatable Read)

SERIALIZABLE (트랜잭션 수준)

스냅샷은 'TX2 시작 시점'에 고정 → t2의 TX1 커밋을 보지 않음
Undo로 CR 블록을 재구성해 30000을 그대로 반환
→ 두 번째 읽기 = 30000 (첫 읽기와 동일 = 반복 읽기 보장)

결과 = (READ COMMITTED 35000, SERIALIZABLE 30000)

핵심은 되감기 기준 시점입니다. READ COMMITTED는 문장마다 시점을 새로 잡아 t3에서 커밋된 35,000을 보고(반복 읽기 불가), SERIALIZABLE은 트랜잭션 시작 시점으로 고정해 Undo로 30,000을 재구성합니다(반복 읽기 보장). 따라서 (35,000, 30,000), 정답은 ①입니다.

②③④는 두 값을 서로 바꾸거나(②) 한쪽 격리수준을 다른 쪽처럼 취급해(③④) 만든 오답입니다.

선택지	판정	이유
①	○	READ COMMITTED는 t3 문장 시점의 커밋 값 35,000, SERIALIZABLE은 트랜잭션 시작 시점으로 고정해 Undo로 30,000을 읽습니다
②	×	두 값을 뒤바꾼 것입니다. 커밋값 35,000을 보는 쪽이 READ COMMITTED, 시작 시점 30,000을 고정해 보는 쪽이 SERIALIZABLE입니다
③	×	SERIALIZABLE을 문장 수준처럼 취급했습니다. 트랜잭션 시작 스냅샷에 고정되므로 t2의 커밋(35,000)이 아니라 30,000을 봅니다
④	×	READ COMMITTED를 트랜잭션 수준처럼 취급했습니다. 매 문장이 시점을 새로 잡으므로 t3에서는 커밋된 35,000을 봅니다

▪ 이 문제의 핵심

1. 오라클은 두 격리수준 모두 MVCC로 비블로킹 읽기를 하며, 차이는 읽기 일관성의 기준 시점(스냅샷 SCN) 에 있습니다.
2. READ COMMITTED = 문장 수준. 매 SELECT가 문장 시작 시점의 커밋 값을 새로 읽어, 두 읽기 사이 커밋을 반영합니다(Non-Repeatable Read 허용).
3. SERIALIZABLE = 트랜잭션 수준. 트랜잭션 시작 시점 스냅샷을 내내 고정해 Undo 기반 CR 블록으로 같은 값을 반복해 읽습니다.
4. 되감을 기준 시점만 다를 뿐, 과거 버전을 Undo로 재구성하는 CR 메커니즘은 동일합니다.

▪ 한 줄 정리 오라클 READ COMMITTED는 문장 시점으로 되감아 커밋된 35,000을, SERIALIZABLE은 트랜잭션 시작 시점으로 고정해 Undo로 30,000을 읽으므로 (35,000, 30,000)인 ①이 적절합니다.

문제 20. 정답 ②

네 행을 두 세션으로 갈라 보유(GRANT)와 요청(WAIT)의 방향을 이으면 고리가 닫힙니다.

SID 61 : (주문 5001) X GRANT → 5001 보유(막는 중)
 (주문 6002) X WAIT → 6002 요청(대기)
 SID 74 : (주문 6002) X GRANT → 6002 보유(막는 중)
 (주문 5001) X WAIT → 5001 요청(대기)

61 → 74가 보유한 6002 를 기다림

74 → 61이 보유한 5001 를 기다림

= 서로가 서로를 기다리는 순환 대기(Deadlock)

②는 SQL Server의 잠금 동작을 정반대로 뒤집었습니다. 잠금 에스컬레이션은 한 문장이 너무 많은 잠금을 획득하면(대략 수천 건) 행→페이지→테이블로 잘게→굵게 올려 잠금 관리 부하를 줄이는 것입니다 — ②가 말한 "테이블→페이지→행으로 낮춰 동시성을 높인다"는 방향이 거꾸로입니다. 게다가 에스컬레이션은 잠금 개수에 반응하는 메커니즘일 뿐, 교착을 분해하거나 해소하는 기능이 아닙니다. 이 교착은 에스컬레이션으로 저절로 풀리는 것이 아니라, 락 모니터가 감지해 한쪽을 희생자로 롤백(오류 1205)해야 풀립니다(③이 옳게 서술). 두 가지 — 에스컬레이션 방향과 "저절로 해소" — 가 모두 반대라, 부적절한 진술은 ②입니다.

①③④는 보유·요청 방향과 순환(①), 자동 감지·희생자 롤백(③), 접근 순서를 맞추면 단순 블로킹으로 그친다는 회피책(④)을 모두 옳게 서술합니다.

선택지	판정	이유
①	○	61은 5001을 GRANT로 보유하고 6002를 WAIT로 요청, 74는 대칭이라 순환 대기가 성립합니다. 표의 네 행 그대로입니다
②	×	에스컬레이션은 행→페이지→테이블로 굵게 올리는 것이지 테이블→행으로 낮추는 게 아니며, 교착을 분해·해소하지도 않습니다. 방향과 효과가 모두 반대입니다
③	○	SQL Server 락 모니터는 교착을 자동 감지해 롤백 비용이 적은 쪽을 victim으로 롤백(오류 1205)하고 상대를 진행시킵니다
④	○	두 세션이 5001→6002 같은 순서로 접근했다면 순환이 안 생겨, 뒤 세션이 앞 세션 커밋까지 잠깐 기다리는 단순 블로킹에 그칩니다

■ 이 문제의 핵심

- 교착(Deadlock)은 두 세션이 서로 보유한 자원을 맞요청할 때 성립합니다. **sys.dm_tran_locks**에서 양쪽이 한 자원은 GRANT, 상대 자원은 WAIT인 모습이 그 징표입니다.
- 잠금 에스컬레이션은 행→페이지→테이블로 굵어지는 방향입니다(잠금 개수 급증에 대한 대응). 동시성을 높이려 잘게 낮추는 것이 아니며, 교착과는 별개 메커니즘입니다.
- SQL Server 락 모니터는 교착을 자동 감지해 롤백 비용이 적은 쪽을 deadlock victim으로 선정, 오류 1205와 함께 롤백해 해소합니다 — 수동 KILL을 기다리지 않습니다.
- 접근 순서를 동일하면(모두 5001→6002) 순환이 깨져 교착이 단순 블로킹으로 완화됩니다.

■ 한 줄 정리 잠금 에스컬레이션은 행→페이지→테이블로 굵어지는 방향이고 교착을 해소하는 기능이 아니며, 교착은 SQL Server 락 모니터가 자동 감지해 희생자를 롤백(1205)하므로 "에스컬레이션이 교착을 저절로 푼다"는 ②가 부적절합니다.